

WEB3 工程师学习指南

以太坊与 EVM

执行引擎、字节码、Pectra 与 Fusaka

Web3 Engineer Guide

版本 V1.0 · 2026.05

供个人学习使用 · MIT License

目录

01	目录	.
02	0 学习目标	.
03	1 直觉先行: 从一笔 USDC 转账反推整个 EVM	.
04	2 升级时间线: 从 Frontier 到 Fusaka 与 Glamsterdam	.
05	3 账户与交易	.
06	4 Gas 模型	.
07	5 EVM 内部: 栈机的六个数据区	.
08	6 Opcodes 全解: 按类讲透	.
09	7 Precompiles 全表	.
10	8 状态树: MPT 现在 → Verkle 将来	.
11	9 客户端谱系: 执行层与共识层逐个看	.
12	10 升级机制: 硬分叉是怎么开会开出来的	.
13	11 工程实现: viem / Foundry / cast 实战	.
14	12 真实交易 trace 逐步走读(PC by PC)	.
15	13 ABI 与函数选择器深入	.
16	14 事件与 logs(receiptsRoot 与 logsBloom)	.
17	15 代理模式与 DELEGATECALL 工程	.
18	16 区块构建链路: mempool → builder → relay → proposer	.
19	17 EVM 字节码逆向工程	.
20	18 常见坑(Top 12)	.
21	19 AI 在 EVM 工作流的位置	.
22	20 习题(含完整解答)	.

23	22 EOF 深入：被剔除又复活的 EVM 重整方案	·
24	23 Verkle Trees 深入：原理、过渡与 EIP-6800	·
25	24 State expiry / History expiry / 弱无状态	·
26	25 Lean Ethereum：beam chain 与后量子安全	·
27	26 Reth 内部架构与 Execution Extensions (ExEx)	·
28	27 Engine API：执行层与共识层之间的对话	·
29	28 USDC 字节码 walkthrough (FiatTokenV2_2)	·
30	29 Pectra 一年后的现场观察	·
31	30 延伸阅读	·

时间锚点 2026-04: Pectra(2025-05-07)、Fusaka(2025-12-03, 含 BPO1 12-09 与 BPO2 2026-01-07)已上线; Glamsterdam 在 2026 H1 末/Q3 候选; Hegota(Verkle)规划在 2026-H2 ~ 2027。操作码速查以 evm.codes 为准。

前置: [模块 02 — 区块链原理与共识](#)。后置: [模块 04 — Solidity 开发](#)。

CHAPTER 01

目录

- [0 学习目标](#)
 - [1 直觉与历史: 为什么 EVM 长这样](#)
 - [2 升级时间线: 从 Frontier 到 Fusaka 与 Glamsterdam](#)
 - [3 账户与交易](#)
 - [4 Gas 模型](#)
 - [5 EVM 内部: 栈机的六个数据区](#)
 - [6 Opcodes 全解: 按类讲透](#)
 - [7 Precompiles 全表](#)
 - [8 状态树: MPT 现在 → Verkle 将来](#)
 - [9 客户端谱系: 执行层与共识层逐个看](#)
 - [10 升级机制: 硬分叉是怎么开会开出来的](#)
 - [11 工程实现: viem / Foundry / cast 实战](#)
 - [12 真实交易 trace 逐步走读\(PC by PC\)](#)
 - [13 ABI 与函数选择器深入](#)
 - [14 事件与 logs\(receiptsRoot 与 logsBloom\)](#)
 - [15 代理模式与 DELEGATECALL 工程](#)
 - [16 区块构建链路: mempool → builder → relay → proposer](#)
 - [17 EVM 字节码逆向工程](#)
 - [18 常见坑\(Top 12\)](#)
 - [19 AI 在 EVM 工作流的位置](#)
 - [20 习题\(含完整解答\)](#)
 - [22 EOF 深入: 被剔除又复活的 EVM 重整方案](#)
 - [23 Verkle Trees 深入: 原理、过渡与 EIP-6800](#)
 - [24 State expiry / History expiry / 弱无状态](#)
 - [25 Lean Ethereum: beam chain 与后量子安全](#)
 - [26 Reth 内部架构与 Execution Extensions \(ExEx\)](#)
 - [27 Engine API: 执行层与共识层之间的对话](#)
 - [28 USDC 字节码 walkthrough\(FiatTokenV2_2\)](#)
 - [29 Pectra 一年后的现场观察](#)
 - [30 延伸阅读](#)
-

CHAPTER 02

0 学习目标

读完这一章，你应该能在面试白板前不慌、在审计 PR 上能挑刺、在 Discord 里看见 ACD 缩写不再装懂。具体长这样：

1. 白板画 EOA → CA 调用栈，标出 `stack/memory/storage/calldata/transient/code` 边界，标出 `CALL/STATICCALL/DELEGATECALL` 差异。
 2. 用 `viem 2.43.3` 在 Sepolia 手动构造、签名、发送 Type 2(1559)和 Type 4(7702)交易。
 3. 识别函数选择器分发表，用 `evm.codes disassembler` 推断函数签名。
 4. 推导一段汇编的 gas，含 `SSTORE` 冷热、`memory` 二次项、`CALL 63/64`、`blob gas`。
 5. 一句话讲清 Pectra、Fusaka 干了什么；讲清 Verkle 与无状态客户端。
 6. 看懂 AllCoreDevs 纪要、EIP 阶段、EOF 为何被剔除。
-

CHAPTER 03

1 直觉先行：从一笔 USDC 转账反推整个 EVM

1.1 一笔交易必须答三个问题

你刚在钱包里按下“转 1000 USDC 给 Alice”。手机震了一下，二十秒后她那边收到通知。中间这二十秒，全世界几千台机器同时干完了一件事——它们都用一模一样的方式把你账户的 1000 USDC 划给了 Alice，谁都没有作弊的余地。

这是 EVM 真正想解的问题。把它拆开，协议必须在 12 秒内答完三个问题：

1. 谁付钱？怎么从一串字节恢复到付款人地址，并扣他的 ETH？
2. 谁执行？一段 16 KB 的字节码该按什么规则跑？跑多久才停？
3. 谁记账？跑完后哪些状态被改了、改成了什么、所有节点凭什么相信你？

每个问题逼出一个 EVM 设计选择。把这三条线拉直，整个模块的内容是它们的展开——你读到的每个 opcode、每张 gas 表，都是为了在这三个问题上不掉链子。

问题	EVM 选择	见
怎么签 + 扣钱	secp256k1 + EIP-2718 typed envelope; EIP-1559 base fee 烧 + 小费	\$3, \$4
怎么停机 + 怎么算	栈机 + 每 opcode 付 gas + 超额 revert	\$5, \$6
怎么记 + 怎么核对	账户模型 + Merkle Patricia Trie + 全节点重算 stateRoot	\$3.1, \$8

1.2 四个硬约束塌缩出 EVM 的形状

EVM 的样子不是某个天才在白板上画出来的，是被四条铁规一路逼出来的。把这四条记住，你看后面那 150 条 opcode 时就不会觉得它们在乱来——它们每一条都是在哪条铁规下不得不那样写。

约束	它逼出的设计
全节点必须算出完全相同的状态	32 字节定长字、整数运算、无浮点
必须停机	每条指令付 gas，超额回滚
状态必须可承诺、可批量验证	Merkle Patricia Trie; header 写 stateRoot
合约不能互相污染对方的存储	每次 CALL 开新栈帧; storage 按合约地址分桶

栈机不是哲学选择，是约束的副产品。每条 opcode 对栈高度的影响是确定的，所以给一个入口高度，编译器就能静态算出整段代码的栈变化——jump 合法性、gas 计费、字节码大小都因此变简单。代价是没有寄存器分配，重复读栈得靠 DUP/SWAP 一通体操，写多了你会怀念 x86。

1.3 六组对偶贯穿全文

EVM 的复杂性，绝大多数时候归结成"两个看起来很像的东西到底有什么区别"。把下面这六组对偶记下来——你读到任何新概念，都先往这六个槽位上塞，它会告诉你"这玩意儿到底站在哪一边"：

- 1. EOA vs CA — 谁能签 vs 谁只能被叫
- 2. memory vs storage — 帧内重置 vs 进 stateRoot
- 3. CALL vs DELEGATECALL — 切上下文 vs 借上下文(代理的根)
- 4. base fee vs priority fee — 烧 vs 给提议者
- 5. execution gas vs blob gas — 两套独立市场
- 6. MPT vs Verkle — 现在 vs 将来

1.4 EVM 在 VM 谱系里的位置

EVM 不是唯一一种"区块链上的虚拟机"。Solana 跑 SVM, Polkadot 跑 WASM, Aptos / Sui 跑 MoveVM, Starknet 跑 CairoVM——每一个都在做差不多的事(停机 + 共识 + 状态承诺)，但选了不同的妥协方案。把它们放在一起对比，你能更清楚 EVM 当初为什么这样选。

VM	类型	状态模型	项目	与 EVM 比
EVM	栈式	account + MPT	以太坊全家	字段定长、生态最大
WASM (eWASM)	栈式	自定	Polkadot、 Soroban	通用、字节码大；ETH 已搁置
SVM	寄存器 (BPF)	account + 平铺	Solana	并行执行强
MoveVM	寄存器	resource	Aptos、Sui	编译期防 double-spend
CairoVM	多项式	Merkle	Starknet	STARK 原生

OP / Arbitrum / Base / zkSync Era / Linea / Scroll / Polygon zkEVM 都以 EVM 为执行层。

CHAPTER 04

2 升级时间线：从 Frontier 到 Fusaka 与 Glamsterdam

EVM 不是一次设计完的。它是被一拳一拳打出来的形状：DAO 被偷过 360 万 ETH，矿工反复用同一段 opcode 拖慢全网，Parity 钱包一夜间冻结 51 万 ETH，rollup 把 calldata 撑到了协议层都开始喘——每一次硬分叉的背后，都是某个真人某天在凌晨三点对着 etherscan 骂街。

读这一节的正确姿势，是每个版本都问一句“它在收拾哪个烂摊子”。

2.1 Frontier(2015-07-30)

第一版主网。CALL/CREATE/SSTORE 已存在但 gas 表过松(CALL 40 gas)，状态膨胀严重。留下 EVM 雏形与 Yellow Paper 第一版。

2.2 Homestead(2016-03-14)

- EIP-2: contract creation gas 升到 32000；私钥 s 值规范化(防 malleability)
- EIP-7: 引入 DELEGATECALL——代理模式的祖先；CALLCODE 开始弃用(msg.sender 语义不对)

2.3 The DAO 分叉(2016-07-20, 区块 1,920,000)

以太坊历史上最尴尬也最关键的一次分叉。

The DAO 是当时全世界最大的众筹合约，募了 1150 万 ETH(≈\$1.5 亿，2016 年的 1.5 亿)。它的 splitDAO 函数干了一件经典蠢事：先把钱转给你，再扣你的余额。攻击者发现这个顺序后，做了个会回头再调一次 splitDAO 的合约——钱还没扣，又能再领一份；递归下去，他抽走了 360 万 ETH。

社区分裂成两派：一派说“代码即法律，认栽”，另一派说“我们是人不是教条”。最后硬分叉把那笔钱回滚了，没回滚的那条链留下来叫 **Ethereum Classic(ETC)**，回滚的那条就是今天的 **Ethereum**。这次事件留下的两份遗产至今仍是 Solidity 程序员的肌肉记忆：**Checks-Effects-Interactions** 模式(先改余额再转账)和 **ACD** 公开讨论惯例(核心决策不再黑箱)。

来源：[Gemini: DAO Hack Explained](#)。

2.4 Tangerine Whistle(2016-10-18)+ Spurious Dragon(2016-11-22)

DAO 风波刚平，2016 年 9-10 月又来一拨 DoS：有人反复调用又便宜又重的 IO opcode(SLOAD、EXTCODESIZE 一类)，收一两块钱手续费就能让全网卡半小时。社区只能硬着头皮再发两次紧急分叉。

- EIP-150(Tangerine Whistle)：IO 类 opcode 大幅涨价；引入 63/64 规则——CALL 最多转发当前 gas 的 63/64，剩 1/64 留本地，阻断“1024 层 call depth attack”
- EIP-155(Spurious Dragon)：交易签名加 chainId，防跨链重放

来源：[EIPS/eip-150](#)、[RareSkills: 63/64 rule](#)。

63/64 规则的现实影响: 黄皮书写着栈深度上限 1024, 但因为每层 CALL 都掉 1/64 gas, 实际能跑到的深度大约是 200~340。如果哪份审计报告还在念叨 "1024 stack depth attack", 那作者多半还活在 2016 年——今天根本到不了 1024。

2.5 Byzantium (2017-10-16)

- EIP-196 / EIP-197: bn254 椭圆曲线加法、乘法、配对 precompile(0x06/0x07/0x08), zkSNARK 首次可在主网验证
- EIP-198: modexp precompile(0x05), RSA / ZK 通用
- EIP-211: RETURNDATASIZE / RETURNDATACOPY, 调用方能读完整返回值
- EIP-214: STATICCALL, 强制只读 CALL, view 函数链上保护的基础
- EIP-100: 难度调整变软(基于平均出块时间)

2.6 Constantinople / Petersburg (2019-02-28)

以太坊唯一一次因安全问题被回滚的硬分叉。原本 2019-01-16 要上 Constantinople, 包含 EIP-1283 (一种新版 SSTORE 的 gas 表)。ChainSecurity 在升级前一天发了一篇分析: 这个新表会让 `transfer / send` 那个写死了的 2300 gas 假设失效, 开出一道全新的重入攻击面。

那一晚 ACD 紧急召开, 社区在几小时内决定回滚整个升级。一个半月后 2019-02-28 上 Petersburg, 剔除 EIP-1283, 保留:

- EIP-145: SHL/SHR/SAR 位移指令
- EIP-1014: CREATE2, 确定性合约地址, counterfactual deployment 的根
- EIP-1052: EXTCODEHASH
- EIP-1234: 区块奖励 3 → 2 ETH, 延后难度炸弹

来源: [CoinDesk: Constantinople delay](#)。

教训很硬: 硬分叉前必须完成 invariant 复审。EIP-1283 自己的逻辑是对的, 它错在没去想"主网上跑着的合约都隐式依赖了 2300 gas 这个数字"。生态系统的旧约定可能被你的新规则一把推翻——这是协议升级永远要交的学费。

2.7 Istanbul (2019-12-08) + Muir Glacier (2020-01-02)

- EIP-152: blake2f precompile(地址 0x09), 用于 Zcash 等链跨链
- EIP-1108: bn254 价格大降, 让 ZK 操作更便宜
- EIP-1344: CHAINID opcode(之前要硬编码)
- EIP-1884: 状态访问类 opcode 涨价(SLOAD 200 → 800); 新增 SELFBALANCE
- EIP-2200: 第三版 SSTORE gas 表, 最终版本(基本沿用至今)
- EIP-2384 (Muir Glacier): 再次推迟难度炸弹

2.8 Berlin (2021-04-15)

- EIP-2565: modexp 涨价(之前算大数 RSA 太便宜)
- EIP-2929: 状态访问 opcode 引入 cold/warm 双轨制——同一笔交易内首次访问某个 (address, slot) 收 cold 价(SLOAD 2100、SSTORE +2100 cold), 后续 warm 价(100)

- EIP-2718: Typed Transaction Envelope, 未来一切新交易类型的总框架
- EIP-2930: Type 1 交易 (access list), 可以预热槽位省 gas

2.9 London (2021-08-05)

DeFi Summer 之后, 主网平均 gas price 飙到 200 gwei, 钱包用户每次都得"猜"出价才不会饿死在 mempool。London 改了游戏规则。

- EIP-1559: Type 2 dynamic fee, 引入 base fee (烧毁) + priority fee (小费)
- EIP-3198: BASEFEE opcode
- EIP-3529: refund 上限砍到 gas_used / 5; 移除 SELFDESTRUCT 退款, 终结 GST2 等 gas token 滥用
- EIP-3554: 再推一次难度炸弹

从此 base fee 由协议自动调节, 钱包从"猜 gas price"改成"设 max fee + max priority fee"。再加上 base fee 烧毁 + Merge 后低发行率 (~0.5%/年), ETH 多数时间是净通缩——这是历史上第一次, 主流 L1 的代币会越用越少。

2.10 Arrow Glacier (2021-12-09) + Gray Glacier (2022-06-30)

纯延后难度炸弹的过渡升级。

2.11 The Merge (2022-09-15, Bellatrix + Paris)

7 年准备, 30 秒切换。执行层几乎一行代码不动, 共识层从 PoW 整体切到 PoS——飞机在天上换了发动机, 没人掉下来。

- mining 完全停止; DIFFICULTY opcode → PREVRANDAO (信标链上一个 epoch 的随机数)
- 区块时间: 13 秒 → 精确 12 秒 (slot)
- ETH 发行量下降约 90%, 与 EIP-1559 烧毁结合

来源: [Mainnet Merge Announcement](#)。

2.12 Shanghai / Capella (2023-04-12)

- EIP-4895: 信标链验证者提款 (withdrawal 不再单向锁仓)
- EIP-3651: COINBASE 直接 warm
- EIP-3855: PUSH0 opcode
- EIP-3860: 限制 initcode size

2.13 Cancun / Deneb (Dencun, 2024-03-13)

L2 工程师等了三年的那次升级。2023 年 Optimism、Arbitrum 把 calldata 当 DA 用, 每天要付几百万美元给主网, rollup 用户做一次 swap 还得 \$5。Cancun 一刀切下去, blob 一上线, L2 swap 当天就变成几毛钱。

- EIP-4844: Proto-danksharding, blob 交易 (Type 3), L2 DA 成本下降一个量级
- EIP-1153: transient storage (TSTORE/TLOAD)

- EIP-5656: MCOPY opcode
- EIP-6780: SELFDESTRUCT 弱化——仅"同一 tx 内创建+自毁"才真删, 否则等同 transfer
- EIP-4788: 信标链区块根写进执行层(合约可 trustless 读共识层数据)
- EIP-7516: BLOBBASEFEE opcode

2024 年起 Optimism、Arbitrum 等 rollup 全部迁到 blob, 主网 calldata 让出大半, L2 swap 费从几美元降到几美分。这是 EVM 历史上第一次为某个上层应用(rollup)专门加交易类型——L2 已经从"以太坊的客人"变成"以太坊的房客"了。

2.14 Prague / Electra(Pectra, 2025-05-07 10:05:11 UTC, epoch 364032)

Merge 之后最大的一次升级, 11 个 EIP 一次性塞进来——账户抽象、机构质押、L2 吞吐三件大事一锅端。这之后的以太坊和你 2024 年用过的以太坊已经不是一个东西。

执行层(Prague):

- EIP-7702: SetCode 交易(Type 4), EOA 临时挂 designator code, 任何已有 EOA 都能跑合约逻辑(\$3.5)
- EIP-2537: BLS12-381 系列 precompile(0x0b ~ 0x11, 共 7 个, \$7)
- EIP-2935: 把过去 8192 个区块哈希存进 `0x0000f90827f1c53a10cb7a02335b175320002935` 系统合约。ZK rollup、跨链桥从此不用维护自己的历史 trie
- EIP-7623: calldata "floor" 机制——一笔 tx 收的 gas 不能低于 $21000 + 10 \times (\text{zero_bytes} + 4 \times \text{non_zero_bytes})$ 。压制纯 calldata 滥用, 把"租 calldata 占空间"挤进 blob
- EIP-7691: blob 目标 3→6、上限 6→9
- EIP-2935、EIP-7549(attestation 聚合优化)

共识层(Electra):

- EIP-7251: 单验证者最大有效余额(MaxEB) 32 → 2048 ETH, 机构无需拆数千 validator, attestation 数量减少
- EIP-6110: deposit 由执行层系统合约直接处理, 新质押 4-12 分钟生效(之前 ~12 小时)
- EIP-7002: 执行层触发的 partial / full exit。质押者用普通 EVM 交易就能发起退出, 无需共识层签名

来源: [Pectra Mainnet Announcement](#)、[The Block: Pectra activated with 11 changes](#)、[ethPandaOps: Pectra Mainnet Checklist](#)。

2.15 Fulu / Osaka(Fusaka, 2025-12-03 21:49:11 UTC, epoch 411392)

Pectra 把 blob 拉到 9, 一些跑家庭 staking 的同学已经在 Reddit 抱怨"我家光纤撑不住了"。继续往上抬就得换思路: 节点不再下载所有 blob, 每人只看一部分, 靠数据可用性采样(DAS)一起投票确认数据存在——这就是 PeerDAS, Fusaka 的主菜。

完整 12 个 EIP(来源: [Alchemy: Fusaka Dev Guide to 12 EIPs](#)、[Conduit: Fusaka EIPs Cheat Sheet](#)):

EIP	名字	作用
EIP-7594	PeerDAS	节点只采样部分数据列; header 加 data_column_sidecars 字段
EIP-7642	eth/69 协议	移除 mempool gossip 中的 pre-Merge 字段 (PoW total difficulty 等)
EIP-7823	MODEXP 上限	base/exp/mod 各 ≤ 8192 字节, 防 DoS
EIP-7825	tx gas cap	单笔交易 gas 上限 30M ($\approx$ block gas 一半)
EIP-7883	MODEXP 涨价	基数 + 阶 + 模再涨一档, 与 Berlin EIP-2565 接力
EIP-7892	Blob Parameter Only fork	引入"轻量分叉"机制: 只调 blob 参数不改 EVM, 一行配置即可 (BPO1/BPO2 就靠它)
EIP-7910	eth_config JSON-RPC	标准化客户端报告自己当前激活了哪些 fork 与 EIP
EIP-7917	提议者前瞻确定性	proposer lookahead 进 beacon state, 让 builder 提前知道下个 slot 的 proposer
EIP-7918	blob base fee 下限	把 blob base fee 与执行 gas 挂钩, 防 blob fee 永远在最低 1 wei
EIP-7934	RLP 区块 size 上限	单区块 RLP 序列化后 ≤ 10 MB
EIP-7935	默认 gas limit 60M	默认建议 gas_limit 从 30M 提到 60M (节点共识投票)
EIP-7939	CLZ opcode	Count Leading Zeros, 新增 0x1e opcode, bit 操作硬件友好
EIP-7951	secp256r1 precompile	又名 P-256, 地址 0x100。passkey / WebAuthn 直接上链

EOF 命运: 原计划进 Fusaka 的 EOF (EIP-3540 / 3670 / 4200 / 4750 / 5450 / 6206 / 7480 / 663) 整组被剔除。理由: 工程复杂度高、与 EIP-7702 的交互未充分研究、社区担心过早冻结字节码格式。可能在 Glamsterdam 之后再议。

已激活的 BPO 链:

- BPO1 (2025-12-09): blob 目标 6 $\rightarrow$ 10、上限 9 $\rightarrow$ 15
- BPO2 (2026-01-07): blob 目标 $\rightarrow$ 14、上限 $\rightarrow$ 21

Prysm 事件: 升级一周后, Prysm 的 PeerDAS 数据列处理路径出现一个 race condition, 依赖它的 validator 集体 attestation 掉线, finality participation 一度降到 $\sim 75\%$ ——但链没分叉。其他四个 CL 客户端 (Lighthouse / Teku / Nimbus / Lodestar) 一起把链扛住了。

这是十年来"客户端多样性"这件事最响的一次现场录音: 它不是营销话术, 是真的能在某家挂掉时让网络继续跑。

来源: [Fusaka Mainnet Announcement](#)、[CoinDesk: Ethereum Activates Fusaka](#)、[Cointelegraph: Prysm Bug Knocks Ethereum](#)。

CHAPTER 05

3 账户与交易

回到 §1.1 那笔 USDC 转账——"谁付钱"和"谁记账"两个问题，最终都落在两样东西上：账户(钱在哪、谁有权花)和交易(一次具体的"我要花"动作)。

账户像银行那本户口簿，交易是签好字的转账单。这一节先看账户长什么样，再看以太坊这十年里发明的五种"转账单"格式——每种格式都为某个具体的痛点而生。

3.1 两种账户

address → Account，四字段：

nonce	u64	
balance	u256	(单位 wei, 1 ETH = 1e18 wei)
codeHash	bytes32	(keccak256 of code, EOA 为 keccak256(""))
storageRoot	bytes32	(该账户 storage trie 的根)

分两类：

	EOA	CA
创建方式	算 secp256k1 公钥 → keccak256 后 20 字节	由其他账户的 CREATE/CREATE2 部署
codeHash	空 keccak256("") 或 (Pectra 后) designator hash	部署字节码哈希
能签交易	能	不能
能持有 ETH	能	能
能被调用	能(无代码 → 仅转账; 有 designator → 跑代理代码)	能

Pectra 后的微妙变化：EOA 的 codeHash 字段在挂上 designator 时变成 keccak256(0xef0100 || delegateAddress)。链上看 getCode(eoa) 返回 23 字节 0xef0100 + 20 字节地址。这是 EIP-7702 的硬约定，任何执行层客户端都按此识别。撤销委托(authorization 指向 0x0)后，EOA 的 codeHash 仍是 keccak256(0xef0100 || 0x0...0)，不会变回 keccak256("")。codeHash 字段一旦被 SetCode 改写，就永远是 designator hash，区分"从未委托过"与"曾委托又撤销"需要看 code 内容(指向 0x0 的 designator)而非比对 codeHash 是否等于空哈希。

3.2 五种交易类型

EIP-2718 typed envelope(type_byte || rlp(payload)，Type 0 legacy 除外)：

类型	名字	EIP	引入	关键字段
0x00	Legacy	—	创世	nonce, gasPrice, gas, to, value, data, v, r, s
0x01	Access List	EIP-2930	Berlin	+ chainId, accesslist
0x02	Dynamic Fee	EIP-1559	London	+ maxPriorityFeePerGas, maxFeePerGas
0x03	Blob	EIP-4844	Cancun	+ maxFeePerBlobGas, blobVersionedHashes
0x04	SetCode	EIP-7702	Pectra	+ authorizationList

新类型几乎是上一类型的"加字段": 1559 $\supset$ 2930, 4844 $\supset$ 1559, 7702 $\supset$ 1559 (4844 与 7702 不同分支)。

3.3 EIP-1559 (Type 2) 字段细解

RLP 结构:

```
0x02 || rlp([
    chain_id,
    nonce,
    max_priority_fee_per_gas, # 给提议者的小费 上限
    max_fee_per_gas,        # 用户愿意付的总价 上限
    gas_limit,
    to,                      # 0x 表示部署
    value,
    data,
    access_list,            # 可选预热
    v, r, s                 # 签名 (v 为 0/1)
])
```

实际花费:

```
gas_price = min(max_fee_per_gas, base_fee + max_priority_fee_per_gas)
fee        = gas_price * gas_used
burn       = base_fee * gas_used      # 被烧毁
tip        = (gas_price - base_fee) * gas_used # 给提议者
```

$\text{max_fee_per_gas} < \text{base_fee} \rightarrow$ 交易进不了 mempool。烧毁是协议层直接从 sender 余额扣除、不进任何账户, 等价 ETH 总量减少。

3.4 EIP-4844 Blob 交易 (Type 3) 字段细解

```
0x03 || rlp([
    chain_id, nonce,
    max_priority_fee_per_gas, max_fee_per_gas,
    gas_limit, to, value, data, access_list,
    max_fee_per_blob_gas,      # blob 单价上限
    blob_versioned_hashes,    # 每个 blob 的 KZG commitment 的 versioned hash 列表
    v, r, s
])
```


Blob 数据走 sidecar(共识层 P2P)，约 18 天后删除，versioned hash 永久上链。执行层只能用 BLOBHASH (0x49) 取 versioned hash、用 0x0a precompile 验 KZG 证明。每 blob = 128 KB，每 blob 的 blob gas = 131072；单块 blob 上限随分叉演进(Cancun 6、Pectra 9、Fusaka BPO1 15、BPO2 21)。

3.5 EIP-7702 SetCode(Type 4) 字段细解

EIP-7702 干了一件以太坊七年没人敢干的事：让 EOA 临时长出代码。在 Pectra 之前，"是 EOA 还是合约"是一辈子的事，地址一旦从助记词派生就永远只是个普通钱包，想要 batch、想要社恢、想要 paymaster，对不起，你得换地址重新部署一个合约钱包，把所有资产搬过去。Pectra 之后，你那个用了三年的 MetaMask 地址，也可以"今天我是 Safe，明天我是 Ambire"，地址不变。

```
0x04 || rlp([
  chain_id, nonce,
  max_priority_fee_per_gas, max_fee_per_gas,
  gas_limit, to, value, data, access_list,
  authorization_list,      # 关键新字段
  v, r, s
])
```

authorization_list 数组，每元素是 EOA 自签的小授权：

```
[chain_id, address, nonce, y_parity, r, s]
```

含义："我 (EOA) 授权把 code 指向 address"。chain_id = 0 表示跨链有效(极不推荐)。

执行流程：

1. 验证发送者签名，收手续费
2. 对每条 authorization: 恢复 EOA 地址 → 校验 chain_id 与 nonce → 写 code = 0xef0100 || address, EOA nonce +1
3. 像普通 1559 跑 to 调用

authority 不必是 sender——任何人可替你代发，只要你签了 authorization(gas sponsor 核心机制)。Pectra 之后，"我帮你付 gas"不再是 4337 的特权。

安全提醒，最重要那条：authorization 一旦上链就生效，签向恶意合约就等于把整个 EOA 双手奉上。Pectra 上线两周内就出现钓鱼合约，有报告说前一个月里 97% 的 authorization 关联钓鱼。永远只签向审计过的合约——这是底线，没有例外。

来源：[EIPS/eip-7702](https://eips.ethereum.org/eip-7702)、ethereum.org/roadmap/pectra/7702。

3.6 ERC-4337(账户抽象)vs EIP-7702

两者不互斥，经常组合：

维度	ERC-4337	EIP-7702
类型	应用层(合约)	协议层(硬分叉)
钱包形态	独立合约钱包, 地址 $\neq$ EOA	改造已有 EOA, 地址不变
Mempool	独立的 4337 mempool	走主网 mempool
入口	EntryPoint 合约(v0.7)	协议直接处理
谁付 gas	Paymaster 或自己	sender(可以是任何人)

实际工程: EOA 通过 EIP-7702 委托到 4337 兼容合约, 拥有 4337 全部能力同时保持 EOA 地址。

CHAPTER 06

4 Gas 模型

Gas 是 EVM 的心跳。每一条 opcode、每一字节 calldata、每一 KB memory 扩展，都得先付钱再跑——这就是 §1.2 那条“必须停机”的铁规怎么落地的。

读这一节时记住一件事：**gas** 公式从来不是为了让用户付钱，是为了让攻击者付不起。每一次涨价、每一次冷热分离、每一次 refund 上限，背后都有一个具体的 DoS 故事。

4.1 基本费用结构

```
total_fee    = gas_used × gas_price
gas_price    = min(max_fee_per_gas, base_fee + max_priority_fee_per_gas)
burn         = base_fee × gas_used
miner_tip    = (gas_price - base_fee) × gas_used
```

base fee 调整算法(EIP-1559):

```
parent_gas_target = parent_gas_limit / ELASTICITY (ELASTICITY = 2)
if parent_gas_used > target:
    delta = parent.base_fee × (parent_gas_used - target) / target / 8
    base_fee = parent.base_fee + max(delta, 1)
else if parent_gas_used < target:
    delta = parent.base_fee × (target - parent_gas_used) / target / 8
    base_fee = parent.base_fee - delta
else:
    base_fee = parent.base_fee
```

每块最多 $\pm 12.5\%$ 。block gas_limit $\approx 30M$ (ELASTICITY=2, 上限 60M)。

4.2 Yellow Paper 的 gas 公式(节选)

Yellow Paper 用 $C(\sigma, \mu, A, I)$ 表示某状态下下一步指令的 gas 消耗。多数 opcode 是常数；特殊家族：

```
C_SSTORE    = 复杂状态依赖 (见 §4.3)
C_SLOAD     = warm 100 / cold 2100 + EIP-2929
C_CALL      = 700 base + value 转账加价 + new account 加价 + 63/64 rule
C_CREATE    = 32000 base + 200 × code_size + memory_expansion
C_LOG_n     = 375 base + 8 × len(data) + 375 × n
C_mem(a)    = 3 × a +  $\lfloor a^2/512 \rfloor$  ; a = 单位 32 字节字
```

C_mem 二次项：前 22 个字(约 700 字节)基本线性，到几 MB 要数百万 gas。

4.3 SSTORE 大表(EIP-2929 + EIP-3529, $\text{refund} \leq \text{gas_used} / 5$)

旧值 → 新值	warm	cold (追加 +2100)	refund
0 → 0	100	2200	0
0 → X(新写)	20000	22100	0
X → 0(清零)	2900	5000	+4800
X → Y(X ≠ 0, Y ≠ 0, Y ≠ X)	2900	5000	0
X → X(等价无操作)	100	2200	0

读:

操作	warm	cold
SLOAD	100	2100
TLOAD(EIP-1153)	100	100(无 cold/warm 概念)
TSTORE	100	100

4.4 内存 expansion 的二次项

EVM memory 惰性扩展:

```
C_mem(a) = 3a + ⌊a² / 512⌋
expansion_cost = C_mem(new_size) - C_mem(old_size)
```

大小	a(32 字节字)	gas
1 KB	32	98
32 KB	1024	5120
1 MB	32768	~2.2M

来源: [Yellow Paper 等式 326](#)、ethereum.org/developers/tutorials/yellow-paper-evm。

4.5 CALL 的 63/64 规则与 stipend

```
forward_gas = min(requested, available × 63/64)
```

EIP-150 把这一条加进来防"call depth 攻击"——你最多把当前 gas 的 63/64 转给被调方, 剩下 1/64 必须留在自己这层。深度每加一层就掉 1/64, 递归到 200~340 层就没 gas 跑了, 攻击者再怎么嵌套也炸不到 1024。

CALL / CALLCODE 带 value 时额外 +9000 gas, 外加给被调方一个 2300 gas stipend——历史上是怕你 fallback 里写了点东西突然没钱跑而 revert。STATICCALL / DELEGATECALL 不带 value, 没有这层礼物。

但这个 2300 gas 早就坑过太多人。Istanbul(EIP-1884)把 SLOAD 涨到 800 之后, 2300 gas 连读两次 storage 都不够, 更别说 EIP-2929 又把 cold SLOAD 涨到 2100。今天还在用 address.transfer(x) 或 address.send(x) 的代码全是定时炸弹——永远别硬编码 gas 常数, 老老实实写 call{value:x} ("") 加 checks-effects-interactions 才是 2026 年的正确姿势。

4.6 transient storage 的 gas(EIP-1153)

TSTORE / TLOAD 固定 100 gas, 无 cold/warm, tx 结束清零, 不入 stateRoot。典型用途: re-entrancy guard(22100 → 100 gas)、一次性 approve+transferFrom、闪电贷簿记。

来源: [Solidity 0.8.24 transient storage](#)。

4.7 blob gas(独立费用市场)

```
blob_base_fee = fake_exponential(MIN_BASE_FEE_PER_BLOB_GAS, excess_blob_gas,
BLOB_BASE_FEE_UPDATE_FRACTION)
fee_per_blob = blob_base_fee × 131072
```

fake_exponential 用泰勒展开模拟 e^x (避免浮点)。BLOB_BASE_FEE_UPDATE_FRACTION = 3338477 (Cancun; Pectra 有调整)。两套费用市场完全独立。

4.8 intrinsic gas

每笔交易执行前先收:

```
21000                # 基础 (普通转账)
+ 32000              # 如果是合约部署
+ 4   per zero byte   of calldata
+ 16  per non-zero byte of calldata   (Pectra 前 16; EIP-7623 后引入"floor"机制)
+ 2400 per access list address
+ 1900 per access list (address, key)
+ PER_EMPTY_ACCOUNT_COST = 25000 per authorization (EIP-7702)
  # 若被授权的 authority EOA 已存在 (绝大多数情况), 退 PER_EMPTY_ACCOUNT_COST -
PER_AUTH_BASE_COST = 12500,
  # 净收 PER_AUTH_BASE_COST = 12500 gas。
```

EIP-7623(Pectra)引入 calldata floor——tx 总 gas 不能低于 calldata floor, 把"租 calldata 占空间"挤进 blob。

CHAPTER 07

5 EVM 内部：栈机的六个数据区

讲完了“账户长什么样、交易怎么签”，下一步该看 EVM 真正运行起来的样子了。一笔交易最终要变成 CALL 进合约 → 跑字节码 → 改状态——这一节回答的就是“跑字节码时，EVM 把数据放在哪、从哪读”。

打开任何一个 EVM 实现的源码，你会看到这台机器其实小得令人吃惊：256 位字长，无寄存器，无浮点，一个栈、一段内存、一些 key-value 表。所有数据落在六个区里。下表盯三列就够：「持久？」决定它进不进 stateRoot（也就是要不要全网都记账，gas 贵不贵），「谁能写」决定隔离边界，「生命周期」决定一个值能活多久。

这六个区是 EVM 的“地理”。后面所有 opcode 都只是在它们之间搬数据。

区	字节寻址？	持久？进 stateRoot？	谁能写	生命周期	主要 opcode
stack	否(按栈位)	否	当前帧	当前调用帧	PUSH, POP, DUP, SWAP
memory	是	否	当前帧	当前调用帧	MLOAD, MSTORE, MSTORE8, MCOPY, MSIZE
storage	否(256 位 slot)	是	当前合约自己	永久	SSTORE, SLOAD
transient storage	否(256 位 slot)	否(不进 stateRoot)	当前合约自己	当前 tx	TSTORE, TLOAD
calldata	是	—	调用方传入	当前帧(只读)	CALLDATALOAD, CALLDATACOPY, CALLDATASIZE
code	是	是	部署时一次写定(EOA Pectra 后例外)	永久	CODECOPY, CODESIZE, EXTCODECOPY, EXTCODESIZE, EXTCODEHASH

5.1 stack

最大 1024 项，每项 32 字节。所有算术与控制流从栈取值、把结果推回栈。

- PUSH1..PUSH32: 推 1~32 字节字面量
- PUSH0(Shanghai 起): 推零
- POP: 丢弃
- DUP1..DUP16: 复制栈顶第 1~16 项
- SWAP1..SWAP16: 交换栈顶与下方第 1~16 位

栈过深触发 "Stack too deep" 编译错误(编译时报, 非运行时)。

5.2 memory

线性字节数组, 初始空, 按 32 字节对齐惰性扩展, 帧结束销毁(不跨 CALL 帧共享)。Solidity 约定 0x40 为 free memory pointer(非协议强制)。MCOPY(EIP-5656)比循环 MLOAD/MSTORE 省约 1/3。

5.3 storage

每个合约都有自己独立的 storage trie, 32 字节 key 配 32 字节 value。Solidity 把状态变量按声明顺序映射到 slot 0、1、2...; mapping 用 `keccak256(key || slot)` 算槽位(这就是为什么 mapping 没法被遍历——你只能算出某个 key 对应的 slot, 没法从 slot 反推回 key)。

SLOAD 冷读 2100, SSTORE 全新写一次冷 22100。gas 优化第一条铁律: 减少 SSTORE。这一条比"用 unchecked"、"压缩 struct"重要十倍——一次冷写就够你跑 200 次 ADD 了。

5.4 transient storage

EIP-1153(Cancun), slot KV, tx 结束清零, 不入 stateRoot, 100 gas。Solidity 0.8.24 起原生 `transient` 类型。

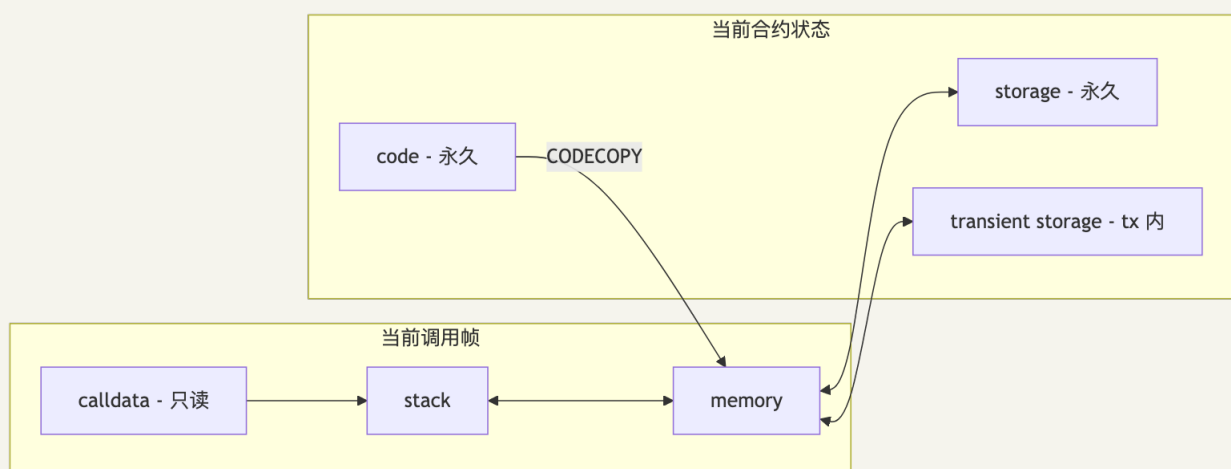
5.5 calldata

调用方传入的只读字节。bytes calldata 避免拷到 memory 省 gas。注意: constructor 参数附在 initcode 末尾, 需 CODECOPY 读, 不走 calldata。

5.6 code

不可变(7702 EOA 例外)。CODECOPY 拷自身代码到 memory; EXTCODECOPY / EXTCODESIZE / EXTCODEHASH 读他人代码。

5.7 一张图把六个数据区串起来(Mermaid)



CHAPTER 08

6 Opcodes 全解：按类讲透

EVM 大约 150 条指令。把它们逐条翻译一遍，这本书就变成了《新华字典》——没人会读字典。所以这一节的安排是：表格只给"形状"(栈进/出、gas 公式、引入分叉)，让你随时能查；然后把那些会咬人的 opcode 单独拉出来讲故事。

哪些会咬人？SELFDESTRUCT 干掉过 51 万 ETH。DELEGATECALL 是一切代理的根，也是一切代理事故的根。tx.origin 让无数钓鱼合约成功。CALLCODE 已经被官方判死刑但还在主网字节码里苟着。这些不能只放在表格里——它们值得每人写一段。

完整速查以 [evm.codes](#) 为准。

读表约定：栈 列写 输入 → 输出，栈顶在右；gas 列省略 memory expansion 与 cold/warm 可变部分；标 **粗体** 的 opcode 在工程实践中容易踩坑或被审计反复盯。

6.1 停机 / 算术 (0x00-0x0B)

编码	助记符	栈	gas	说明
0x00	STOP	→	0	正常停机；不消耗 gas
0x01	ADD	a, b → a+b	3	256 位 wrap
0x02	MUL	a, b → a×b	5	wrap
0x03	SUB	a, b → a-b	3	wrap
0x04	DIV	a, b → a÷b (b=0 返 0)	5	无符号
0x05	SDIV	a, b → a÷b	5	二补码有符号
0x06	MOD	a, b → a mod b	5	无符号
0x07	SMOD	a, b → a mod b	5	有符号
0x08	ADDMOD	a, b, n → (a+b) mod n	8	防中间溢出
0x09	MULMOD	a, b, n → (a×b) mod n	8	同上
0x0a	EXP	a, b → a^b	10 + 50×byte_len(b)	byte_len(b) 是 b 的字节数
0x0b	SIGNEXTEND	b, x → ...	5	x 按 b 字节符号扩展到 32 字节

这一组里有三个反直觉的小坑，每个都坑过有名有姓的合约：

- **DIV / MOD** 在 `b=0` 时返 0，不是 **panic**——这是底层 EVM 的设计，Solidity 0.8 自己往代码里塞了 `if (b == 0) revert();`，才让你以为它会报错。如果你写汇编绕过去了，`0 / 0` 会安静地给你 0。
- **EXP** gas 与指数字节数线性增长：`2 ** N` 当 `N` 很大时巨贵，而 `1 << N` 是 **SHL**，3 gas 一个固定价。算固定二次幂，永远用位移。
- **Solidity 0.8** 之前没有 **checked** 算术，0.8 之后还能用 `unchecked { }` 关掉——审计永远要把每一个 `unchecked` 块单独盯一遍，那是协议作者亲手卸下安全带的地方。

6.2 比较与位运算 (0x10-0x1D)

编码	助记符	栈	gas	说明
0x10	LT	<code>a, b → a < b ? 1 : 0</code>	3	无符号
0x11	GT	<code>a, b → a > b ? 1 : 0</code>	3	无符号
0x12	SLT	<code>a, b → ...</code>	3	有符号
0x13	SGT	<code>a, b → ...</code>	3	有符号
0x14	EQ	<code>a, b → a == b ? 1 : 0</code>	3	—
0x15	ISZERO	<code>a → a == 0 ? 1 : 0</code>	3	—
0x16	AND	<code>a, b → a & b</code>	3	按位
0x17	OR	<code>a, b → a b</code>	3	按位
0x18	XOR	<code>a, b → a ^ b</code>	3	按位
0x19	NOT	<code>a → ~a</code>	3	按位取反
0x1a	BYTE	<code>i, x → x 的第 i 字节（高位优先）</code>	3	<code>i ≥ 32</code> 返 0
0x1b	SHL	<code>shift, value → value << shift</code>	3	EIP-145 (Constantinople)
0x1c	SHR	<code>shift, value → value >> shift</code>	3	同上
0x1d	SAR	<code>shift, value → value >> shift (算术)</code>	3	同上
0x1e	CLZ	<code>x → 前导零位数</code>	5	EIP-7939 (Fusaka)

CLZ(Count Leading Zeros, EIP-7939)：Fusaka 新增，用于 `fast-log2`、定点数实现，硬件友好。

6.3 SHA3 / KECCAK (0x20)

EVM 唯一原生哈希。注意名字这件事——它助记符叫 **SHA3**，但其实算的不是 NIST **SHA-3**，是它的前身 **Keccak-256**。这个错位是 2015 年定标准时的历史遗留，至今还有人因为搞错而签错消息。

编码	助记符	栈	gas	说明
0x20	KECCAK256(旧名 SHA3)	offset, len → keccak256(mem[offset..offset+len])	$30 + 6 \times \lceil \text{len}/32 \rceil + \text{mem_expand}$	EVM 唯一原生哈希

小哈希便宜得令人感动。但不要在循环里 keccak 大 buffer——memory expansion 是二次项，per-word 费用又会叠加上去，循环个十次就能把 gas 干到 50000+。

6.4 环境信息 (0x30-0x3F)

编码	助记符	栈	gas	说明
0x30	ADDRESS	→ this	2	当前合约地址
0x31	BALANCE	addr → balance(addr)	2600 (cold) / 100 (warm)	跨账户读
0x32	ORIGIN	→ tx.origin	2	永远不要用作权限
0x33	CALLER	→ msg.sender	2	直接调用方
0x34	CALLVALUE	→ msg.value	2	wei
0x35	CALLDATALOAD	i → calldata[i..i+32]	3	越界部分填 0
0x36	CALLDATASIZE	→ size	2	—
0x37	CALLDATACOPY	destOffset, offset, len →	$3 + 3 \times \lceil \text{len}/32 \rceil + \text{mem_expand}$	calldata→memory
0x38	CODESIZE	→ size	2	当前合约 code 长度
0x39	CODECOPY	destOffset, offset, len →	$3 + 3 \times \lceil \text{len}/32 \rceil$	自身 code→memory
0x3a	GASPRICE	→ effective_gas_price	2	1559 后可能等于 baseFee+tip
0x3b	EXTCODESIZE	addr → size	2600/100	别人的 code 长度
0x3c	EXTCODECOPY	addr, destOffset, offset, len →	同上 + copy fee	别人 code→memory
0x3d	RETURNDATASIZE	→ size	2	上次 CALL 返回数据长度
0x3e	RETURNDATACOPY	destOffset, offset, len →	$3 + 3 \times \lceil \text{len}/32 \rceil$	RETURNDATA→memory
0x3f	EXTCODEHASH	addr → keccak256(code)	2600/100	EIP-1052

BLOCKHASH 在 Pectra (EIP-2935) 后的语义：这是个极隐蔽的坑。opcode 标价仍然写 20 gas——你看 evm.codes 是这么写的，黄皮书也是这么写的。但 Pectra 之后，客户端实现实际上会去读那个 history 系统合约 (0x0000F90827F1C53a10cb7A02335B175320002935) 的 storage。冷读直接吃一个 SLOAD cold 2100 gas，同一笔 tx 内第二次访问才变 warm。

Gas 估算时不能再当 20 gas，否则你的 gasLimit 会比实际需要少 2080，交易直接 OOG。审计代码看到 BLOCKHASH 都得多看两眼。

6.5 区块信息 (0x40-0x4A)

编码	助记符	栈	gas	说明
0x40	BLOCKHASH	n → hash	20	仅最近 256 块；EIP-2935 后由系统合约提供 8192 (见下方 caveat)
0x41	COINBASE	→ proposer_addr	2	Merge 前是 miner
0x42	TIMESTAMP	→ block.timestamp	2	proposer 可小幅操纵
0x43	NUMBER	→ block.number	2	—
0x44	PREVRANDAO	→ randao mix	2	Merge 后替代 DIFFICULTY
0x45	GASLIMIT	→ block.gaslimit	2	—
0x46	CHAINID	→ chain_id	2	EIP-1344 (Istanbul)
0x47	SELFBALANCE	→ balance(this)	5	比 BALANCE(this) 便宜
0x48	BASEFEE	→ block.baseFee	2	EIP-3198 (London)
0x49	BLOBHASH	i → blob_hashes[i]	3	EIP-4844 (Cancun)
0x4a	BLOBBASEFEE	→ blob_base_fee	2	EIP-7516 (Cancun)

6.6 栈、内存、存储、控制流 (0x50-0x5F)

编码	助记符	栈	gas	说明
0x50	POP	<code>a →</code>	2	丢弃栈顶
0x51	MLOAD	<code>offset → mem[offset.. +32]</code>	3 + mem_expand	—
0x52	MSTORE	<code>offset, value →</code>	3 + mem_expand	写 32 字节
0x53	MSTORE8	<code>offset, value →</code>	3 + mem_expand	仅写 1 字节
0x54	SLOAD	<code>key → storage[key]</code>	100 (warm) / 2100 (cold)	见 §4.3
0x55	SSTORE	<code>key, value →</code>	见 §4.3 大表	最贵的 opcode 家族
0x56	JUMP	<code>dest →</code>	8	dest 必须是 JUMPDEST
0x57	JUMPI	<code>dest, cond →</code>	10	cond ≠ 0 才跳
0x58	PC	<code>→ pc</code>	2	当前 PC, 调试用
0x59	MSIZE	<code>→ memory_size_bytes</code>	2	当前 memory 高水位
0x5a	GAS	<code>→ gasleft</code>	2	剩余 gas
0x5b	JUMPDEST	<code>→</code>	1	合法跳转标记
0x5c	TLOAD	<code>key → transient[key]</code>	100	EIP-1153 (Cancun)
0x5d	TSTORE	<code>key, value →</code>	100	EIP-1153 (Cancun)
0x5e	MCOPY	<code>dest, src, len →</code>	3 + 3 × $\lceil \text{len}/32 \rceil$ + mem_expand	EIP-5656 (Cancun)
0x5f	PUSH0	<code>→ 0</code>	2	EIP-3855 (Shanghai)

JUMPDEST 是"合法跳转目标", 跳到非 JUMPDEST 字节会 revert。这是字节码形式的"函数边界", 也是 disassembler 划分 basic block 的依据。

6.7 PUSH / DUP / SWAP (0x60-0x9F)

- PUSH1..PUSH32 (0x60-0x7F): 推字面量 1..32 字节, 3 gas
- DUP1..DUP16 (0x80-0x8F): 复制栈顶第 1..16 项, 3 gas
- SWAP1..SWAP16 (0x90-0x9F): 交换栈顶与第 2..17 项, 3 gas

PUSH0 (EIP-3855, Shanghai) 省 1 字节, 降合约部署费。

6.8 日志 (0xA0-0xA4)

编码	助记符	栈	gas	说明
0xa0	LOG0	offset, len →	$375 + 8 \times \text{len}$	无 indexed
0xa1	LOG1	offset, len, t1 →	$750 + 8 \times \text{len}$	1 个 indexed topic
0xa2	LOG2	offset, len, t1, t2 →	$1125 + 8 \times \text{len}$	2 个
0xa3	LOG3	offset, len, t1, t2, t3 →	$1500 + 8 \times \text{len}$	3 个
0xa4	LOG4	offset, len, t1..t4 →	$1875 + 8 \times \text{len}$	4 个

LOG 不影响 stateRoot, 只进 receiptsRoot 与 logsBloom, 前端与 indexer 读它。ERC-20 转账用 LOG3(topic = [Transfer 哈希, from, to], data = amount)。

6.9 系统调用 (0xF0-0xFF): 四种 CALL 与 CREATE

整个 EVM 里最危险、最容易出事故、也最值得花时间搞懂的一组 opcode。

EVM 准备了四种 CALL, 不是因为四种好看, 是因为代理模式、库合约、view 函数保护、安全围栏每一个都需要不同的语义。它们的差别全在三件事上: msg.sender 是谁、storage 写到哪、能不能改状态。

先把这张对比表背下来, 比理解任何字节码都重要——很多审计漏洞最后归结成"作者不知道这次 CALL 把 msg.sender 改成谁了"。

Opcode	msg.sender	storage 写到	能转 ETH	状态可写	引入
CALL	当前合约	被调合约	是	是	Frontier
CALLCODE	当前合约	当前合约(!)	是	是	Frontier(弃用)
DELEGATECALL	保持原 caller	当前合约	否(继承 value)	是	Homestead
STATICCALL	当前合约	被调合约	否	否	Byzantium

编码	助记符	栈	gas	说明
0xf0	CREATE	value, off, len → addr	32000 + initcode 执行 + 200 × deployed_size	地址 = keccak256(rlp([sender, nonce]))[12:]
0xf1	CALL	gas, addr, value, ai, as, ro, rs → success	700 + 9000(value ≠ 0) + 25000(new account) + cold + child gas	标准外部调用
0xf2	CALLCODE	同上	同上	已弃用；用 DELEGATECALL
0xf3	RETURN	offset, len →	0 + mem_expand	正常返回
0xf4	DELEGATECALL	gas, addr, ai, as, ro, rs → success	700 + cold + child	保留 sender 与 storage 上下文
0xf5	CREATE2	value, off, len, salt → addr	32000 + 6 × ⌈len/32⌉ + ...	地址 = keccak256(0xff sender salt keccak(initcode)) [12:]
0xfa	STATICCALL	gas, addr, ai, as, ro, rs → success	700 + cold + child	强制只读
0xfd	REVERT	offset, len →	0 + mem_expand	回滚但保留 returndata
0xfe	INVALID	→	all gas	故意非法字节，吃光 gas
0xff	SELFDESTRUCT	recipient →	5000 (cold + 2600)	EIP-6780 后弱化

6.9.1 CALLCODE 为什么死了

CALLCODE 是一个错版本的 DELEGATECALL，活了不到一年就被废了。它把别人的代码搬到自己家跑（storage 用自己的），但 msg.sender 还是写死的当前合约——结果就是任何被搬来的代码做 require(msg.sender == owner) 都报废了，因为它看见的 sender 永远是 proxy 自己，不是真正的用户。

Homestead(2016-03, EIP-7)引入 DELEGATECALL 修了这个: msg.sender 保留为最外层的原 caller, 代理模式才真正成立。CALLCODE 当场进入弃用, 主网上几乎再没人用, 但字节码里它还活着, 反汇编时如果看见 0xf2 就要小心——大概率是 2016 年的老古董。

6.9.2 DELEGATECALL: EVM 最危险的家伙

DELEGATECALL 一句话: 它把别人的代码搬到你家, 用你的钥匙开你的保险箱。

```
proxy.fallback() {
  delegatecall(implementation, calldata)
  return returndata
}
```

implementation 的字节码跑在 proxy 的上下文:

- proxy 的 storage 被读写(implementation 自己的 storage 永远碰不到)
- msg.sender / msg.value 保持用户原值
- 谁是 address(this) ? 是 proxy

这就是 EIP-1967 透明代理、UUPS、Diamond、ERC-4337 钱包、EIP-7702——以太坊半壁江山的"可升级"基础设施都靠这一条 opcode。也是为什么每年被偷走最多钱的那一组漏洞, 都和它有关。

Parity 多签事件(2017-11-06)——一节课价值 5 亿美金:

剧情: Parity 把钱包逻辑放在 library 合约里, 部署一个, 几千个用户的 stub 钱包都 DELEGATECALL 到这个 library。问题是这个 library 自己是个普通合约, 没人调用过它的 initWallet 给它定 owner——它一直裸在主网上, 以为只有 stub 会调它。

直到那天有人路过, 发现:

1. 我可以直接 CALL 这个 library 的 initWallet
2. owner 是空的, 所以我把自己设成 owner
3. 作为 owner, 我可以调 kill
4. SELFDESTRUCT, library 消失

library 一死, 所有指向它的 stub 钱包都成了"DELEGATECALL 到空地址"的废墟。约 51.3 万 ETH(当时 ~\$1.5 亿, 今天接近 20 亿)永久冻结。没人偷走这笔钱, 他们只是把通往保险箱的桥炸了。

来源: [OpenZeppelin: Parity Wallet Hack Reloaded](#)、[smartcontractshacking.com: delegatecall attacks](#)。

三条事故复盘出来的硬规矩:

- 永远 init library 自身——OZ 的 _disableInitializers() 就是为这个生的
- implementation 入口加 require(address(this) != _self), 一旦有人直接调 implementation 立刻 revert
- EIP-6780 后 SELFDESTRUCT 弱化, 但旧合约还跑在主网, 依旧得当心

6.9.3 CREATE vs CREATE2

```
CREATE : keccak256(rlp([sender, nonce]))[12:32]
CREATE2 : keccak256(0xff || sender || salt || keccak256(initCode))[12:32]
```

CREATE 让你的合约地址跟 nonce 走, nonce 一变地址就变——很难"先告诉别人地址, 再部署"。CREATE2 把 nonce 换成你自己选的 salt, 部署前就能算出地址。这一改解锁了三件大事: Uniswap V2/V3 不用维护 pool 地址表了(用 token 对加 fee 当 salt 直接算)、ERC-4337 钱包能 counterfactual 收钱(钱包没部署但地址已经在收 USDC)、Foundry 的 `cast create2` 可以挖虚策地址。

6.9.4 STATICCALL: 链上 view 的硬保护

Byzantium 引进的"只读封印"。STATICCALL 把"我只想看不想动"这件事写进了协议层——一旦一个调用是 STATICCALL, 从这里开始整条调用栈里任何 SSTORE、CREATE、SELFDESTRUCT、LOG 都会 revert。

为什么要这条? 因为 Solidity 的 `view / pure` 修饰符在 view 函数里也允许调外部合约。如果没有 STATICCALL, 被调那边可以悄悄改自己的 storage, 然后你的 view 函数突然就"有副作用"了。STATICCALL 让这件事从依赖好心人变成协议保证。

6.9.5 SELFDESTRUCT 在 Cancun 后的弱化(EIP-6780)

SELFDESTRUCT 这一辈子做过的最有名一件事, 就是把 51 万 ETH 锁在 Parity 多签里至今没解。Cancun 终于决定削它一刀。

EIP-6780 之前: 调用一次 SELFDESTRUCT, 合约的 code、storage 全部清零, 余额转给指定地址。代码彻底消失。

EIP-6780 之后: 只有同一笔 tx 内"创建后立刻自毁"才会真的删除, 其他情况下 SELFDESTRUCT 退化成本普通转账——余额转走, code 和 storage 一动不动。

这条弱化也"杀死"了一种叫 **metamorphic contract** 的炫技模式: 原版做法是 SELFDESTRUCT 把 storage 清掉, 然后用同一个 salt + initCode 调 CREATE2 重新部署一个长得一样但内部状态全新的合约——因为 storage 真的清了。EIP-6780 之后这个把戏失效, 旧 storage 还在那里, "重置后的"合约一上来就读到上一辈子的残留。审计任何号称"可重置"的合约时必须重点核对这条。

6.10 一段汇编反汇编后的可读形态

0x6080604052348015600f57600080fd5b5060358060206000396000f3fe... 反汇编:


```

PUSH1 0x80      ; free memory pointer 初值
PUSH1 0x40      ; Solidity 约定 fmp 槽
MSTORE          ; memory[0x40..0x60] = 0x80
CALLVALUE       ; msg.value
DUP1
ISZERO
PUSH1 0x0f
JUMPI           ; if value == 0 跳到主体
PUSH1 0x00
DUP1
REVERT          ; 否则 revert (构造函数不接受 ETH)
JUMPDEST        ; 0x0f
POP
...

```

这是所有 Solidity 合约的 prelude——分配 fmp、检查 msg.value、部署 runtime code。看到此段即可认定来自 Solidity 编译器，而非手写 Yul。

CHAPTER 09

7 Precompiles 全表

Precompile 是 EVM 的"作弊码"——某些计算如果让 EVM 一条 opcode 一条 opcode 跑, gas 算出来比矿工读心还贵;那就让客户端用 native 代码(C / Rust / Go)原生跑,再贴个固定地址当合约假装。

举个真实数字:用 EVM 字节码跑一次 secp256k1 椭圆曲线签名恢复要几百万 gas,会让链直接停摆。但 0x01 这个 precompile 让你用 3000 gas 就调到 OpenSSL 写好的 native 实现。这就是为什么以太坊从 Frontier 起就硬编码了几个曲线、哈希、配对操作——整个 ZK 生态、跨链桥、ERC-4337、EIP-7702、blob 验证全建在这十几个魔法地址上。

截至 Pectra(2025-05)一共 17 个,地址从 0x01 到 0x11;Fusaka 又给 0x100 留了个 secp256r1。

地址	名字	引入	主要用途	gas 公式
0x01	ecrecover	Frontier	secp256k1 签名恢复	3000
0x02	sha256	Frontier	SHA-256 哈希	$60 + 12 \times \lceil \text{len} / 32 \rceil$
0x03	ripemd160	Frontier	RIPEMD-160	$600 + 120 \times \lceil \text{len} / 32 \rceil$
0x04	identity	Frontier	datacopy(memory 拷贝)	$15 + 3 \times \lceil \text{len} / 32 \rceil$
0x05	modexp	Byzantium / Berlin EIP-2565 涨价	大数模幂(RSA、ZK)	复杂公式, 与位长平方相关
0x06	bn254 G1 add	Byzantium	bn254 加法	150
0x07	bn254 G1 mul	Byzantium	bn254 标量乘	6000
0x08	bn254 pairing	Byzantium	bn254 配对 (zkSNARK)	$45000 + 34000 \times k$
0x09	blake2f	Istanbul	BLAKE2 压缩	$1 / \text{round}$
0x0a	point evaluation	Cancun	KZG 点求值 (4844)	50000(定值)
0x0b	BLS12-381 G1 ADD	Pectra	BLS 加法	375
0x0c	BLS12-381 G1 MSM	Pectra	G1 多标量乘法	复杂(discount 表)
0x0d	BLS12-381 G2 ADD	Pectra	G2 加法	600
0x0e	BLS12-381 G2 MSM	Pectra	G2 多标量乘法	复杂
0x0f	BLS12-381 PAIRING_CHECK	Pectra	BLS 配对检查	$32600 + 37700 \times k$
0x10	BLS12-381 MAP_FP_TO_G1	Pectra	base field $\rightarrow$ G1	5500
0x11	BLS12-381 MAP_FP2_TO_G2	Pectra	extension field $\rightarrow$ G2	23800

来源: [EIPS/eip-2537](https://eips.ethereum.org/EIP-2537)、evm.codes/precompiled。

7.1 ecrecover(0x01)

整个以太坊最常被调用的 precompile, 没有之一——你每签一次 EIP-712 permit、每过一次 4337 UserOp、每验一次链下消息, 背后都是这 32 字节的输出。

输入 128 字节 `hash || v || r || s`，输出 32 字节地址(恢复失败返回 0x0，别忘了检查这个，把全 0 当合法地址是经典漏洞)。

经典坑还有一个：**signature malleability**。secp256k1 的数学性质决定了 (r, s) 和 $(r, n-s)$ 能恢复出同一个地址，攻击者只要把 s 翻一下，就得到一个看起来不同但合法的签名——如果你拿签名哈希做了“防重放”去重，他刚好绕过去。ecrecover 自己不强制 $s \leq n/2$ ，所以你要么手动检查，要么直接用 OpenZeppelin 的 ECDSA.sol。

7.2 modexp(0x05)

$(B^E) \bmod M$ ，big-int。Berlin EIP-2565 收紧 gas 公式(大致是 $\max(Bsize, Msize)^2 \times bit_len(E) / 3$)，与计算量对齐。用于 RSA 验证、ZK 电路。

7.3 bn254 系列(0x06/07/08)

bn254(alt_bn128)。zkSNARK(Groth16)基于此曲线。安全性约 110 位，BLS12-381(128 位)在 Pectra 引入。

7.4 blake2f(0x09)

BLAKE2 的压缩函数(非完整函数)，让 ETH 能验证 Zcash 等用 BLAKE2 的链的区块头。

7.5 point evaluation(0x0a)

EIP-4844 核心。输入 192 字节：`versioned_hash(32) || z(32) || y(32) || commitment(48) || proof(48)`。校验“承诺多项式在点 z 确实等于 y ”。rollup 用 (z, y, proof) 在主网证明 batch 数据完整性，比放 calldata 便宜约 100 倍。

7.6 BLS12-381 系列(0x0b ~ 0x11, EIP-2537, Pectra)

地址	名字	输入 / 输出	gas
0x0b	G1 ADD	256 → 128 字节	375
0x0c	G1 MSM $\sum k_i \times P_i$	$N \times 160 \rightarrow 128$ 字节	discount 表
0x0d	G2 ADD	512 → 256 字节	600
0x0e	G2 MSM	$N \times 288 \rightarrow 256$ 字节	discount 表
0x0f	PAIRING_CHECK $\prod e(P_i, Q_i) = 1$	$N \times 384 \rightarrow 32$ 字节	$32600 + 37700 \times k$
0x10	MAP_FP_TO_G1	64 → 128 字节	5500
0x11	MAP_FP2_TO_G2	128 → 256 字节	23800

曲线安全性 ~128 位。MSM 替代了原草案的分散 G1MUL/G2MUL(BLST/arkworks 标准做法的硬件化)。工程意义：链上验证 Cosmos/Polkadot/Sui 的 BLS 签名、ZK rollup 最终配对验证、信标链 BLS 签名在执行层廉价验证。

7.7 secp256r1 / P-256 (0x100, EIP-7951, Fusaka)

为什么你苹果手机里的指纹和 Face ID 一直没法直接签链上交易？因为它们用的是 secp256r1 (NIST P-256)，而以太坊从 Frontier 起就只会算 secp256k1 (比特币的曲线，少一个字母)，P-256 在 EVM 上得花 ~3M gas 才算得出来。

EIP-7951 一上线，0x100 这个地址直接调 OpenSSL，160 字节进、32 字节出。从此手机原生 passkey 不再需要“先翻译成 ETH 私钥”这种中间层——你的 4337 钱包可以做到“按一下指纹，签一笔链上交易”。这是 EOA 这十年来最接近“没有助记词的钱包”的一次。

7.8 总览记忆口诀

```
0x01-0x04 : 哈希 / 签名 / 拷贝 (Frontier)
0x05      : modexp (Byzantium)
0x06-0x08 : bn254 (Byzantium, ZK Groth16)
0x09      : blake2f (Istanbul, 跨链)
0x0a      : KZG point eval (Cancun, 4844)
0x0b-0x11 : BLS12-381 全家 (Pectra, BLS / 现代 ZK)
0x100     : secp256r1 (Fusaka, passkey)
```

口诀：“1234 哈希拷，5 modexp，678 bn254，9 blake，A KZG，B 起 BLS，100 P-256”。

CHAPTER 10

8 状态树: MPT 现在 → Verkle 将来

每次你查 `cast balance vitalik.eth`，背后有一棵全网都信的树在告诉你答案。

这棵树是以太坊"所有人对状态达成共识"的工具：每个全节点本地都维护它，新块来时按相同规则更新，最后算出的 root 必须 byte-by-byte 一致——只要这个 32 字节的 root 一样，全世界几千台机器对账户余额、合约 storage、code 的看法就完全一致。这是无信任的物理基础。

但这棵树也是以太坊最大的肿瘤之一：350 GB 状态、单证明 1 KB、深度 7-9 层、扇出只有 16。Verkle Trees 是给它做的"心脏外科手术"，在 Hegota 升级后落地。

8.1 Merkle Patricia Trie(现行)

Patricia trie + Merkle(节点 RLP 序列化后 keccak256 当指针) = MPT。四种节点：Null / Leaf(剩余路径, value) / Extension(共享前缀, child) / Branch(16 子节点 + 可选 value)。

实际四棵 MPT: State trie(address → Account)、每合约 storage trie、Transactions trie、Receipts trie，四 root 均进区块头。

痛点：全节点状态约 350 GB(2026-04)；单槽 Merkle 证明 ~1 KB，数百槽即数百 KB，无状态客户端不可行；扇出 16，典型路径深 7-9 层。

8.2 Verkle Trees(将来)

承诺方案：Pedersen + 椭圆曲线(Banderwagon over BLS12-381)。关键参数：扇出 256、证明 ~150 字节(可批量聚合，证 1000 个 slot 共 ~10 KB)、树深 3-4。一个区块的 witness ~200 KB，任何人能用一块 + witness 在轻硬件上验证状态转移(无状态客户端)。代价：椭圆曲线运算比 keccak 慢，节点 CPU 升高。

2026-04 状态：Verkle 测试网(kaustinen 系列)迭代多轮，geth/reth/besu 均有 Verkle 分支。预计 Hegota(2026-H2 ~ 2027)落主网。

来源：ethereum.org/roadmap/verkle-trees、[The State of Ethereum in 2026 — stake.fish](https://stake.fish)。

8.3 一个工程师视角的对比表

维度	MPT(现行)	Verkle(将来)
节点扇出	16	256
哈希	keccak256	Pedersen + 椭圆曲线
单证明大小	~1 KB	~150 字节
批证明聚合	不支持	支持, $O(\log n)$
平均深度	7-9	3-4
单节点 CPU	低(纯哈希)	高(曲线运算)
状态可"无"持有?	否	是
工程成熟度	极高	测试网迭代中

CHAPTER 11

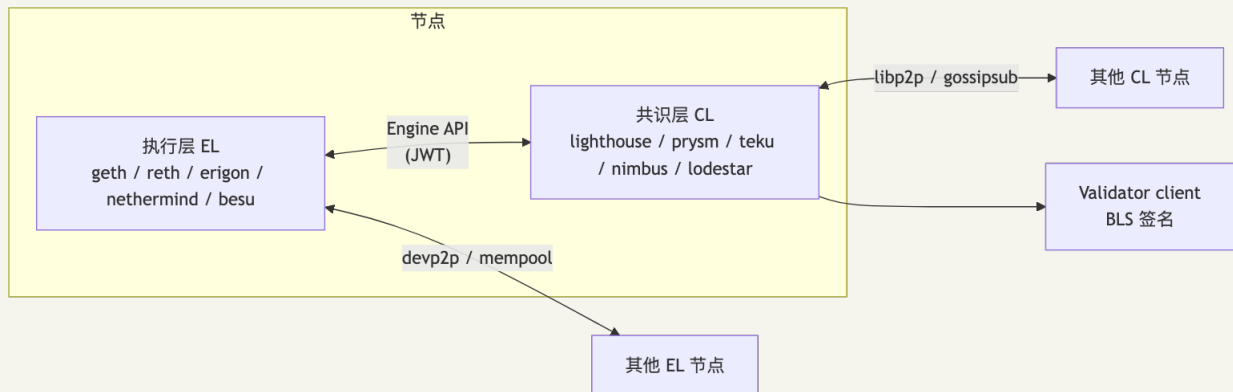
9 客户端谱系：执行层与共识层逐个看

以太坊不是一个程序，是十个程序。

每台全节点其实是两个进程在合作：一个 EL(执行层)跑 EVM、维护状态；一个 CL(共识层)跑 PoS、和别的节点投票。它们用 **Engine API**(HTTPS + JWT)打电话。EL 这边五家：geth / reth / erigon / nethermind / besu；CL 那边五家：lighthouse / prysm / tekum / nimbus / lodestar—— $5 \times 5 = 25$ 种组合，每一种都能跑出一个合法的以太坊节点。

为什么搞这么麻烦？因为 \$2.15 那次 Prysm 事件给的答案最直白：当某一家挂掉时，其他四家可以把链扛住。这是 Ethereum 这十年最隐藏也最关键的工程哲学——共识不是把所有人塞进一个仓库，是让多个仓库相互监督。

9.1 一图看清架构(Mermaid)



9.2 EL: go-ethereum (geth)

- 作者：以太坊基金会，Go
- 版本(2026-04)：v1.16.7 "Ballistic Drift Stabilizer", 含 BPO1/BPO2 mainnet 配置
- 特点：最老最完整，新分叉语义规范以 geth 行为为准；v1.16 起 path-based archive(PBSS)
- 份额：曾长期 80%+, 2025 年起降至 50% 上下
- 文档：geth.ethereum.org/docs

9.3 EL: reth (Paradigm)

- 作者：Paradigm, Rust
- 版本(2026-04)：v2.1.0；v2.0 默认启用 Storage V2(MDBX 重写)，1.7 Gigagas/s, 磁盘 ~240 GB；reth db migrate-v2 无须 resync
- 特点：模块化(EVM、状态 DB、RPC 均可独立用作 crate)；衍生 [ress](#) (无状态客户端原型)
- 份额(2026-04)：约 20-25%，2025 年起快速增长
- 文档：reth.rs

9.4 EL: erigon

- 作者: erigontech, Go
- 版本(2026-04): v3.x; 扁平 KV(plain state), 存档节点省 ~50% 存储; v3 起内嵌 Caplin(CL)做单进程节点
- 注意: erigon 2 已于 2025-05-01 停机, 必须迁 v3
- 文档: docs.erigon.tech

9.5 EL: nethermind

- 作者: Nethermind, C# / .NET
- 特点: 可配置性强; Fusaka 12 个 EIP 中 Nethermind 工程师 co-author 7 个; 多数 staking-as-a-service 平台默认
- 文档: github.com/NethermindEth/nethermind/releases

9.6 EL: besu

- 作者: Hyperledger / Consensus, Java
- 特点: 企业取向(QBFT / IBFT、隐私交易、permissioned); 状态库 Bonsai Tries(扁平 + 压缩)
- 文档: besu.hyperledger.org

9.7 CL: lighthouse (Sigma Prime)

- 作者: Sigma Prime, Rust; v8.0.0+(2026-04), 支持 PeerDAS、`--supernode` / `--semi-supernode`
- 特点: 低内存、稳定, home staker 常选
- 文档: github.com/sigp/lighthouse/releases

9.8 CL: prysm (Offchain Labs)

- 作者: Offchain Labs, Go; 曾长期 CL 份额第一
- 2025-12 事件: Fusaka 上线后 ~7 天 attestation 处理 bug, 依赖 Prysm 的 validator 掉线, finality 降至 ~75%; 其他四客户端维持了链。此后 lighthouse/teku 份额各涨 ~5%
- 文档: github.com/OffchainLabs/prysm/releases

9.9 CL: teku (Consensus)

- 作者: Consensus, Java; Infura 内部使用; JVM 生态友好
- Fusaka 后移除 `GOSSIP_MAX_SIZE` / `MAX_CHUNK_SIZE` / `TTFB_TIMEOUT` / `RESP_TIMEOUT` 配置项
- 文档: github.com/Consensus/teku/releases

9.10 CL: nimbus (Status)

- 作者: Status, Nim; v25.11.0+; 极致省资源, 可跑树莓派全验证者; `--light-supernode` 配合 PeerDAS
- 文档: github.com/status-im/nimbus-eth2

9.11 CL: lodestar (ChainSafe)

- 作者: ChainSafe, TypeScript; v1.36.0+; 唯一 TS/JS 共识客户端, 浏览器友好
- 文档: github.com/ChainSafe/lodestar/releases

9.12 客户端选型决策表

你是	推荐 EL	推荐 CL	理由
Home staker	reth / nimbus	lighthouse / nimbus	资源轻、稳定
大型 staking 平台	nethermind / erigon	teku / lighthouse	可观测性 + JVM 生态
RPC provider (Alchemy 等)	reth / geth	lighthouse	性能 + 历史 archive
研究 / 实验	reth	lodestar / nimbus	模块化 + 可读源码
企业 / permissioned	besu	teku	Java + 隐私功能
移动 / 嵌入式	(等 stateless / Verkle)	nimbus	资源极小

黄金规则：优先选份额 < 33% 的客户端——见 §29.5 Prysm 事件复盘。

CHAPTER 12

10 升级机制：硬分叉是怎么开会开出来的

以太坊没有 CEO。没人有“按下去”的红色按钮。Pectra 那 11 个 EIP 怎么塞进去的、Prague EOF 是谁拍的板要剔除？没人。是几百个开发者每两周开一次 Zoom，吵两小时，慢慢吵出来的。

如果你来自传统软件公司，会觉得这个过程慢得离谱——但它能跑十年没出过一次因决策黑箱而引发的链分裂。这一节把这个看似松散的流程拆开，让你听懂下次 ACD 会议为什么会争得面红耳赤。

10.1 EIP 流程

```

Idea → Draft → Review → Last Call → Final
                        ↓
                    Withdrawn / Stagnant
  
```

每个 EIP 一个 markdown 文件，存在 github.com/ethereum/EIPs。Status: Final ≠ 上主网——还要进硬分叉的 meta EIP 列表（如 [EIP-7600 Pectra Hardfork Meta](#)）。

10.2 ACD (AllCoreDevs)

ACDE（执行层）和 ACDC（共识层）每两周各开一次，Zoom + YouTube 直播，议程：EIP 纳入清单、客户端进度、devnet 结果、testnet 日期。会议纪要：github.com/ethereum/pm。另有 ACDT 专门追踪当前硬分叉的实现进度。

10.3 测试链路

```

local devnet → public devnet (devnet-N) → testnet (Holesky / Hoodi / Sepolia) → mainnet
  
```

- Holesky (2023-09 起)：大规模 PoS / 验证者测试
- Hoodi (2025 起)：Pectra/Fusaka 长寿命测试网，Pectra 在此首发
- Sepolia：开发者日常用，轻量
- Goerli：已退役 (2024-12)

10.4 devnet 节奏

每次分叉上测试网前约经历 10 个 devnet 周期 (devnet-0 → ... → devnet-N)。任何客户端在某 devnet 无法跟住，对应 EIP 可能延至下次分叉 (EOF 在 Fusaka 被剔除的直接原因之一)。

10.5 Fusaka 后的节奏

Fusaka 上线时 ACD 决议：大致春季 + 秋季各一次硬分叉。Glamsterdam 春季款，Hegota 秋季款。

来源：[The Block: twice-a-year hard-fork schedule](#)。

CHAPTER 13

11 工程实现: viem / Foundry / cast 实战

到这里你已经被规则塞了八节, 是时候把这些字打到键盘上。

每个示例都对应前面一个公式: 1559 那一节的"加密签名再 RLP"会落到 `serializeTransaction + keccak256 + signTransaction` 三行; SSTORE 冷热表会变成 `forge test -vvvv` 里你能亲眼看见的 gas 数字; CREATE2 那个 `0xff || ... || keccak(initCode)` 公式会缩成 `cast create2` 一句话。

读不动公式没关系, 跑一次代码大概率比读三遍正文管用。

环境锁定: Node.js 20.18.0 LTS / viem 2.43.3 (2026-02) / Foundry stable v1.0.x 或 nightly 2026-04-25+ / Sepolia RPC: <https://ethereum-sepolia-rpc.publicnode.com>

来源: [Foundry v1.0 Announcement](#)、[viem releases](#)。

11.1 viem 读链 — `code/01-read-mainnet.ts`

`createPublicClient` + 公开 RPC: 读最新区块、ETH / USDC 余额、估 gas、查 1559 fee。

```
cd 03-以太坊与EVM/code
pnpm i && pnpm run read
```

11.2 viem 手撸 1559 — `code/02-sign-and-send-1559.ts`

```
SEPOLIA_PRIVATE_KEY=0xabc... pnpm run send
```

代码展示: `serializeTransaction()` (EIP-2718+1559 编码) → `keccak256()` (secp256k1 实际签的消息) → `account.signTransaction()` (本地签名) → `sendRawTransaction()` → 按 `gasUsed × baseFee` 算烧毁 ETH。

11.3 viem EIP-7702 — `code/03-eip7702-delegate.ts`

```
SEPOLIA_PRIVATE_KEY=0xabc... pnpm run delegate
```

跑完去 sepolia.etherscan.io 看你的 EOA 的 code 字段, 应该是 `0xef0100...`。viem 文档: [Sending Transactions with EIP-7702](#)。

11.4 cast 速查 — `code/04-cast-recipes.sh`

12 个配方: 查余额、解码 calldata、估 gas、计算 keccak、解析 storage slot、CREATE2 地址预言、读 EOA designator。挑感兴趣的段落复制粘贴, 勿直接 `bash 04-...`。

11.5 Foundry 跑 EVM trace — code/05-foundry-bytecode/

```
cd code/05-foundry-bytecode
forge install foundry-rs/forge-std --no-commit
forge test -vvvv --match-contract StackPlayTest
```

三个 demo: bump() (SSTORE cold/warm)、transientPlay() (TSTORE/TLOAD)、mcopyPlay() (MCOPY)。-vvvv 打印每步 opcode trace。

11.6 evm.codes Playground

evm.codes/playground 粘贴:

```
60 06 PUSH1 0x06
60 07 PUSH1 0x07
02 MUL ; stack: [42]
60 00 PUSH1 0x00
55 SSTORE ; storage[0] = 42
00 STOP
```

点 Run 逐步显示 stack / storage。替换 02 为 01/04/07 等过一遍算术 opcode。

CHAPTER 14

12 真实交易 trace 逐步走读(PC by PC)

第一节那笔"转 1000 USDC 给 Alice"的交易,到这里终于能完整还原了。

如果你顺着前面看到这里,应该已经攒了一肚子"这玩意儿到底怎么连起来"的问题: RLP envelope 长什么样? selector 怎么分发? 栈机一步步怎么动? 哪一步触发 SSTORE 冷热? DELEGATECALL 的时候 msg.sender 真的会变吗? LOG3 怎么进 receiptsRoot? 答案全在那 60000 gas 里——这一节就把这 60000 gas 按 PC(程序计数器)走一遍,每一步停下来看栈、看 memory、看 storage 是怎么变的。

读完这一节,你下次 `cast run` 一个 trace 时,就不再是看天书。

调用形式: EOA → USDC proxy → DELEGATECALL → FiatTokenV2_2.transfer。tx 哈希示意:

`0x4d61c79e21b81df9...`, 可用 `cast run --quick` 拉自己的 trace。

12.1 交易解码

`cast tx 0x4d61...` 输出大致:

```
type:          eip1559
nonce:         123
to:            0xA0b8...USDC
value:         0
data:          0xa9059cbb000000...0000003b9aca00
gasLimit:      65000
maxFeePerGas:  25 gwei
maxPriorityFeePerGas: 2 gwei
```

`0xa9059cbb` = `bytes4(keccak256("transfer(address,uint256)"))` (ERC-20 transfer selector)。后 32 字节 = to 地址(前 12 字节填 0), 再 32 字节 = amount = `0x3b9aca00` = `1e9` = 1000 USDC(6 位小数)。

12.2 USDC 合约的 fallback / dispatcher(简化)

```

PUSH1 0x80          ; init free memory pointer
PUSH1 0x40
MSTORE
CALLVALUE           ; msg.value
DUP1
ISZERO
PUSH2 0x000f
JUMPI               ; if value == 0 跳到主体
PUSH1 0
DUP1
REVERT              ; 否则 revert (USDC 不接受 ETH)
JUMPDEST [0x000f]
POP
PUSH1 0x04          ; 取 calldata 的前 4 字节
CALLDATASIZE
LT                  ; calldata < 4 ?
PUSH2 0x0094        ; 跳到 fallback
JUMPI
PUSH1 0
CALLDATALOAD
PUSH1 0xe0
SHR                 ; 把高 4 字节移到低位 (取出 selector)
DUP1
PUSH4 0xa9059cbb    ; 比对 transfer
EQ
PUSH2 0x00b9
JUMPI               ; 命中跳到 transfer 实现
... (其他 selector 比较) ...
JUMPDEST [0x00b9]
... transfer 实现 ...

```

栈每一步的状态完全确定——这就是 disassembler 的工作，看着字节流推算栈和 PC，把 EVM 在脑子里跑一遍。

12.3 PC-by-PC: dispatcher 的 24 步演示

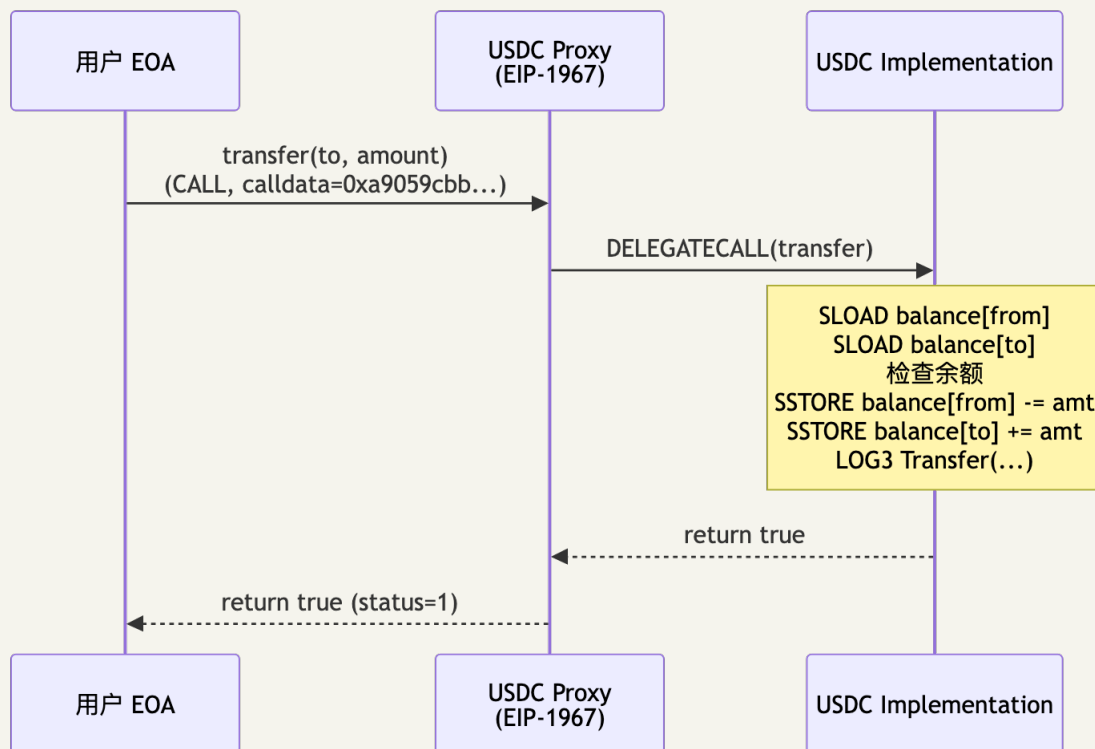
假设 calldata = 0xa9059cbb || 0x...vitalik || 0x...3b9aca00 (76 bytes), msg.value = 0。

PC	字节	opcode	执行后 stack	memory / storage
0x00	60 80	PUSH1 0x80	[0x80]	—
0x02	60 40	PUSH1 0x40	[0x80, 0x40]	—
0x04	52	MSTORE	[]	mem[0x40..0x60] = 0x80 (free memory pointer 初值)
0x05	34	CALLVALUE	[0]	—
0x06	80	DUP1	[0, 0]	—
0x07	15	ISZERO	[0, 1]	—
0x08	61 00 0f	PUSH2 0x000f	[0, 1, 0x0f]	—
0x0b	57	JUMPI	[0] (跳走, 因栈顶 1 truthy)	PC = 0x0f
0x0f	5b	JUMPDEST	[0]	—
0x10	50	POP	[]	—
0x11	60 04	PUSH1 0x04	[4]	—
0x13	36	CALLDATASIZE	[4, 76]	—
0x14	10	LT	[0] (4 < 76 = false)	—
0x15	61 00 94	PUSH2 0x0094	[0, 0x94]	—
0x18	57	JUMPI	[] (不跳走, 因栈顶 0)	—
0x19	60 00	PUSH1 0x00	[0]	—
0x1b	35	CALLDATALOAD	[0xa9059cbb000000...]	取 calldata[0..32]
0x1c	60 e0	PUSH1 0xe0	[0xa9059cbb..., 224]	—
0x1e	1c	SHR	[0x00000000a9059cbb]	右移 224 位提出 selector
0x1f	80	DUP1	[0xa9059cbb, 0xa9059cbb]	—
0x20	63 a9 05 9c bb	PUSH4 0xa9059cbb	[0xa9059cbb, 0xa9059cbb, 0xa9059cbb]	—
0x25	14	EQ	[0xa9059cbb, 1]	—
0x26	61 00 b9	PUSH2 0x00b9	[0xa9059cbb, 1, 0xb9]	—
0x29	57	JUMPI		PC = 0xb9

PC	字节	opcode	执行后 stack	memory / storage
			[0xa9059cbb] (跳到 transfer 实现)	

注意一件事: dispatcher 一步 SSTORE 都没有, 所有动作都在栈上——所以一笔 USDC 转账 60000 gas 里, dispatcher 通常只占 200-500 gas。便宜得不像话, 但每笔交易都要走一遍。这就是为什么 EOF 的 EIP-4750(CALLF/RETF 函数 ABI)省的就是这一段。

12.4 Mermaid 调用栈



USDC 用 EIP-1967 透明代理——实际执行地址与用户调用地址不同, 但 storage 永远是 proxy 的 (DELEGATECALL 语义)。

12.5 transfer 内部 gas 实拆

- SLOAD balance[from] (cold, 2100 gas)
- 检查余额够不够 (GTE); 不够 revert
- SLOAD balance[to] (cold, 2100 gas)
- SSTORE 新 from balance (X→Y, cold, 5000 gas, 无 refund)
- SSTORE 新 to balance: 原来是 0 → "0→X 新写", 22100 gas; 已有余额则 5000
- LOG3: emit Transfer(from, to, amount), 约 1875 gas (375 base + 3×375 topic + 8×32 data)
- RETURN bool true

对一笔"双方都已经持有 USDC"的 transfer, 链上 gas 大致:

```

21000 (intrinsic)
+ 4   × N_zero   + 16 × N_nonzero (calldata ~ 700)
+ 2 SLOAD      (4200 cold)
+ 2 SSTORE     (5000 + 5000)
+ 1 LOG3       (~ 1875)
+ misc opcodes (~ 5000)
≈ 47000 gas

```

实测 USDC transfer 约 41000-65000 gas, 与估算同数量级。

12.6 receipts 与 logsBloom

receipt 里能看到:

- gasUsed
- effectiveGasPrice = min(maxFee, baseFee + maxPriority)
- logs (被 indexer 抓走)
- status = 1 (成功) 或 0 (revert)
- type = 2 (EIP-1559)
- logsBloom : 256 字节布隆过滤器, 把每个 log 的 address 与每个 indexed topic 哈希后映射进 bit 向量。indexer 用它做"快速过滤无关 tx"

跑 `cast receipt 0x4d61...` 后塞给 [Phalcon Explorer](#) 或 [Tenderly](#) 看完整可视化 trace。

12.7 Foundry 复现这次 trace

```

# fork 主网到调用块的前一刻
anvil --fork-url $ETH_RPC_URL --fork-block-number $BLOCK_NUMBER

# 另开一个终端
cast run 0x4d61... --rpc-url http://127.0.0.1:8545 --quick
# 输出 trace tree, 能逐层看到 CALL / DELEGATECALL / SLOAD / SSTORE / LOG

```

`--quick` 跳过状态预热; `-vvv` 打开详细 trace; `--debug` 启动单步调试器(forge debug 风格)。

CHAPTER 15

13 ABI 与函数选择器深入

为什么一个 ERC-20 合约有几十个函数，`calldata` 上来只有 4 字节就能告诉它你要调哪个？

因为这 4 字节是函数签名的 keccak256 前 4 字节——

`bytes4(keccak256("transfer(address,uint256))) = 0xa9059cbb`，全宇宙只有它对应 ERC-20 的 `transfer`。这玩意儿叫 **selector**，是 Solidity 编译器和 EVM 之间的接头暗号。

13.1 selector 的来历

函数选择器 = `bytes4(keccak256(canonical_signature))`，`canonical_signature` 是函数名 + 参数类型规范写法(无空格，`uint` 必须写成 `uint256`)。

```
cast sig "transfer(address,uint256)" # → 0xa9059cbb
cast sig "balanceOf(address)"       # → 0x70a08231
cast sig "approve(address,uint256)" # → 0x095ea7b3
```

13.2 ABI encoding 三规则

1. 静态类型右侧填 0 至 32 字节
2. 动态类型(`bytes`、`string`、`T[]`)放在尾部，头部写 `offset`；`offset` 单位是字节，从参数列表起始算
3. 数组先 32 字节 `length`，后元素

实战例子：`transfer(address to, uint256 amount)`：

```
0xa9059cbb // selector
000000000000000000000000d8da6bf26964af9d7eed9e03e53415d37aa96045 // to (左 12 字节填 0)
000000000000000000000000000000000000000000000000000000000000003b9aca00 // amount = 1e9
```

13.3 selector clash

4 字节是 32 位，理论碰撞概率 $1/2^{32}$ ，但生日悖论 + 真实合约里函数名总集中在那么几个动词，碰撞真的会发生。已经有过几例：`burn(uint256)` 和另一个看起来风马牛不相及的函数共享同一个 `selector`。

Diamond(EIP-2535)这种"多 facet 路由"模式必须严格检查唯一性，否则两个 facet 想注册同一个 `selector`，路由就分不清调谁。`forge inspect Contract methodIdentifiers` 一次性把整个合约的 `selector` 列出来，肉眼扫一遍。

13.4 Solidity ABI 与 packed 编码差异

`abi.encode` 和 `abi.encodePacked` 的区别已经送走过几个百万级合约。

- `abi.encode`：标准 ABI，每参数独立 32 字节，永远不会歧义

- `abi.encodePacked`：紧凑拼接，没 padding。绝对不要拿来当哈希签名参数——
`keccak256(abi.encodePacked("a","bc"))` 和 `keccak256(abi.encodePacked("ab","c"))` 算出来一模一样，这是经典签名重放攻击。

EIP-712 之所以要求 `abi.encode` + 类型哈希一层一层串联，就是为了从源头堵死 packed 歧义。如果你看见任何项目用 `encodePacked` 做签名 message，关掉项目页面就行，不用客气。

13.5 calldata 优化技巧

- `bytes calldata` 替代 `bytes memory`，省 `calldata`→`memory` 拷贝
 - 长度固定时用 `bytes32` 替代 `bytes memory`
 - ABI v2 默认开启 (Solidity 0.8+)，支持 nested struct
-

CHAPTER 16

14 事件与 logs(receiptsRoot 与 logsBloom)

打开 Etherscan 任何一笔交易，下面那一长串"Transfer"、"Approval"、"Swap" 是怎么来的？

它们不在 stateRoot 里，链不会"看见"它们；它们只在 receiptsRoot 里，专门给链下读：The Graph、Subsquid、你的 Dune dashboard、MetaMask 的那个交易历史——所有"链下知道链上发生了什么"的能力，都靠 LOG opcode 系列 + logsBloom 这套基础设施。

14.1 LOG opcode 系列

- LOG0 (0xa0)：无 indexed topic，只有 data
- LOG1 (0xa1)：1 个 topic(通常是事件签名 hash)
- LOG2 (0xa2)：2 个 topic
- LOG3 (0xa3)：3 个 topic
- LOG4 (0xa4)：4 个 topic(事件最多 3 个 indexed 参数 + 事件签名)

gas: $375 \text{ base} + 8 \times \text{len}(\text{data}) + 375 \times n$ (n 个 topic)。

14.2 事件签名 hash

```
event Transfer(address indexed from, address indexed to, uint256 value);
// topic[0] = keccak256("Transfer(address,address,uint256)")
//           = 0xddf252ad1be2c89b69c2b068fc378daa952ba7f163c4a11628f55a4df523b3ef
// topic[1] = from (address padded to 32 bytes)
// topic[2] = to
// data     = abi.encode(value)
```

anonymous 事件没有 topic[0]，indexer 检索时只能用其它 topic 过滤。Solidity 里写 `event Foo() anonymous;` 即声明。

14.3 logsBloom

256 字节布隆过滤器：对 (address, 每个 topic) 哈希后取 3 个 11 位偏移置位。轻节点拉一块即 O(1) 判断"可能含我感兴趣的 log"。进 receiptsRoot 和 block header，Etherscan logs 查询、viem `getLogs` 依赖此结构。

14.4 indexer 工程

- [The Graph](#)：subgraph, GraphQL
- [Subsquid](#)：高性能替代
- [Goldsky](#)、[Envio](#)：商用
- 自建：eth_getLogs + PostgreSQL / ClickHouse

CHAPTER 17

15 代理模式与 DELEGATECALL 工程

15.1 为什么需要代理

合约一旦部署，字节码永久冻结。这是 EVM 的物理事实——你部署完了发现一个 typo? 对不起，那段错的字节码会永远活在主网。

但现实业务不可能不升级。USDC 要加合规接口、Aave 要换利率模型、4337 wallet 要支持新签名算法——怎么办？

代理模式给的答案：把“地址”和“逻辑”拆开。proxy 是个永久不变的合约，里面只做一件事：“收到调用，DELEGATECALL 转给 implementation”；所有用户的 storage、balance、approve 都在 proxy 里；implementation 只放“这次的逻辑”。升级 = 改 proxy 里那个指向 implementation 的指针。

代价你已经在 §6.9.2 见过了：DELEGATECALL 是 EVM 最危险的家伙。代理模式让你能升级，也让 Parity 51 万 ETH 一夜冻结。这一节就是教你怎么用它而不被它咬。

15.2 EIP-1967: 标准化 storage slot

EIP-1967 规定伪随机 slot，避免 proxy 与 implementation 变量撞槽：

```
implementation slot = bytes32(uint256(keccak256("eip1967.proxy.implementation")) - 1)
                    = 0x360894a13ba1a3210667c828492db98dca3e2076cc3735a920a3ca505d382bbc
admin slot          = bytes32(uint256(keccak256("eip1967.proxy.admin")) - 1)
beacon slot         = bytes32(uint256(keccak256("eip1967.proxy.beacon")) - 1)
```

keccak("...") - 1：减 1 把它推出 keccak 像空间，mapping 计算无法碰撞到它。

15.3 三种主流代理形态

- **TransparentProxy(OZ)**: admin 走 admin path, 普通用户走 fallback → DELEGATECALL。slot 防撞清晰，每次调用多 ~2k gas
- **UUPS(EIP-1822)**: 升级逻辑在 implementation, proxy 极简只 fallback。代价: implementation 必须包含 `_authorizeUpgrade`，否则永久不能再升级
- **Diamond(EIP-2535)**: 多 facet 按 selector 路由，复杂但适合大型协议

15.4 Beacon proxy

信标合约持有 implementation 地址，多 proxy 共享同一 beacon。一次更新 beacon = 全部 proxy 升级。Compound、Aave 常用。

15.5 Storage layout 升级金科玉律

- 只追加新变量，不改现有变量类型和顺序
- 不在父合约删 / 重命名变量

- 用 OZ `__gap` 占位留 50 个 slot

15.6 DELEGATECALL 安全清单

- `disableInitializers()` 锁死 implementation, 防止直接调用 + `selfdestruct`
 - 入口加 `require(address(this) != _self) (onlyProxy)`
 - 升级前 `forge inspect` 比对 storage layout
 - 用 OpenZeppelin Upgrades 插件自动检查
-

CHAPTER 18

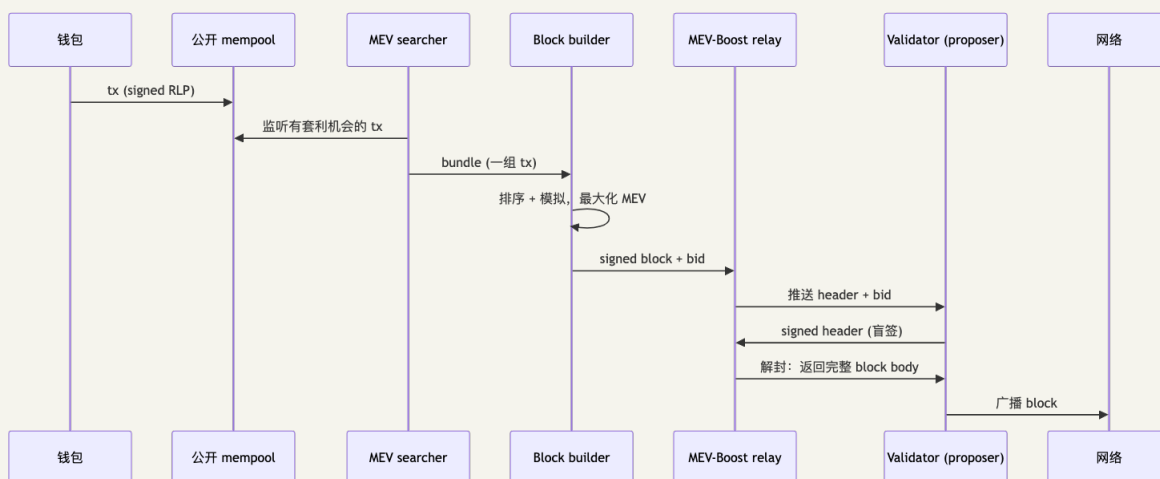
16 区块构建链路: mempool → builder → relay → proposer

你按下"发送", 到这笔交易真的进区块, 中间发生了什么?

短答: 你扔进 mempool, 几个搜索者(searcher)盯着它找套利机会, 几个 builder 把它和其他交易打包成一个候选块, relay 把"最有钱"那个块的 header 推给当前 slot 的 proposer, proposer 盲签后广播——大约 200ms 到 4s 之间, 你的交易就上链了。

长答见下文。这条链路是 MEV 的诞生地、抢跑攻击的舞台、ePBS 想杀掉的中间人——理解它, 你才能理解为什么有人愿意付几千万美元去当 builder, 为什么 Flashbots Protect 这种"私有 mempool"的产品突然冒出来。

16.1 区块生命周期



16.2 公共 mempool vs 私有 mempool

公开 mempool: 广播给所有节点, 搜索者可见, 容易被 sandwich。

私有保护方案: Flashbots Protect / MEV Blocker (直发 builder)、Alchemy / QuickNode MEV-protected RPC。

16.3 ePBS (EIP-7732, Glamsterdam)

proposer 只盲签 commitment, builder 在共识层独立公开 block body——去掉 relay 的信任依赖, 压缩 MEV 操控空间。

16.4 一个 tx 在每个阶段可能花费的时间

- 签名后入 mempool : 即刻
 - builder 看到 : 100-300 ms
 - builder 模拟入候选块 : 1-3 s
 - proposer 接受 bid : 在 slot 开始前 200 ms
 - block 广播 : slot 内 4 s 之前
 - 2 epochs 之后 finality : ~12.8 minutes (32 slot × 12s × 2)
-

CHAPTER 19

17 EVM 字节码逆向工程

链上有一个看着很可疑的合约。它没在 Etherscan 上 verify 源码，每天有几万 ETH 进出，开发者钱包里突然出现了 1000 ETH 不知所踪——你想知道它在干什么。

字节码逆向就是这种时候的活儿。它不是科学，是手艺：先识别 selector 表分清“它有几个函数”，再反查每个 4-byte 找出函数名，再追踪每段 jump 重建控制流，最后在脑子里把 Solidity 写回来。这一节给你工作流和工具。

17.1 工作流

1. 获取 bytecode: `cast code <address>` 或 Etherscan
2. 识别 selector 表: 找 `0xe0 SHR + EQ + JUMPI` 模式
3. 反查每个 4-byte selector: [4byte.directory](#) 或 `cast 4byte`
4. 跟踪 JUMP 目标, 逐 PC 模拟
5. 重建 Solidity 等价

17.2 工具

- [evm.codes Playground](#) — 浏览器内 EVM
- [whatsabi](#) — 字节码 → ABI 推断
- [Sevm](#) — 字节码 → IR
- [Heimdall-rs](#) — 反编译为伪 Solidity
- [Panoramix](#) — 老牌反编译, Etherscan 上的 "Decompile" 来源
- [Dedaub](#) — 在线高质量反编译, 对 OZ 模板识别强

17.3 字节码常见模式

模式	出现位置	含义
<code>608060405234801561...</code>	合约头	Solidity prelude (init free mem ptr + check msg.value=0)
<code>6080604052348015...600080fd5b50...</code>	constructor 头	等价 <code>if (msg.value != 0) revert();</code>
<code>7f<32 bytes>...EQ</code>	dispatcher	EIP-712 domain hash 检查
<code>60a060405234801561</code>	合约头 (带 immutable)	constructor 把 immutable 写入
<code>0x360894a13ba1a3...</code>	proxy 合约	EIP-1967 implementation slot
<code>5b6000fd5b...</code>	function 边界	JUMPDEST + STOP

17.4 对照练习

找 Etherscan 上一个未 verify 的合约: `cast code` → `cast 4byte` 反查 → [Dedaub](#) / Heimdall 反编译
→ 让 Claude 按"PC by PC + stack"格式 (§19.3) 逐段解释 → `forge test vm.prank` 验证重建 ABI。

CHAPTER 20

18 常见坑(Top 12)

每条都有名有姓的祭品。按"被坑过的人 × 被偷的钱"排序：

1. DELEGATECALL 到 storage layout 不一致的合约 — Parity 级灾难
 2. tx.origin 做权限 — 钓鱼必过
 3. 签名 malleability — 无限 s 值, (r,s) 与 (r,n-s) 恢复同一地址
 4. 跨链重放 — chainId 未编进 signed message
 5. 整数溢出 — 0.8 前; 现多见于 unchecked 块
 6. 未初始化 implementation — Parity 另一面
 7. 重入 — checks-effects-interactions 被破坏
 8. 依赖 block.timestamp — 验证者可小幅操纵
 9. 依赖 BLOCKHASH — 仅最近 256 块
 10. 假设 1024 stack depth — 实际 ~340
 11. transfer/send 假设 2300 gas stipend — 现已无法完成任何有意义操作
 12. 未删 SELFDESTRUCT — EIP-6780 后危害降低但非零
-

CHAPTER 21

19 AI 在 EVM 工作流的位置

不是所有事都该让 AI 干。EVM 这种"一个字符错就吃光 gas"的环境，AI 表现得很两极：模板代码反编译它能 80 分，gas 优化最后那 5% 它一定瞎编。

知道哪些事让它做、哪些事别让它做，就是这一节。

19.1 用 AI 能省时间的场景

- 逆向字节码：推断函数签名、识别 OZ / Uniswap V2-V4 / AAVE 模板，常见模板准确率高；混淆 (inlining、selector 乱序) 会掉点
- 解释 calldata：Etherscan、Phalcon Explorer、Tenderly Tx Trace 均自带 AI 旁注
- 生成 ABI：未 verify 合约配合 whatsabi / Sevm 更稳
- 审计副驾驶：Cyfrin Aderyn、OZ Defender、Slither 的 LLM 解释器；Phalcon AI Investigator 给套利/攻击摘要
- gas 优化 hint：指出 SSTORE 0→X 能否合并

19.2 AI 帮不了的场景

ACD trade-off 决策、形式化 invariant 证明 (Halmos / Certora / Kontrol)、gas 优化最后 5-10% (人眼盯字节码仍有效)、全新加密原语设计。

19.3 让 AI 解释字节码的正确 prompt

这一条我专门写一段，因为太多人翻车在这。

错的提法： 把这段字节码翻译成 Solidity。AI 会幻觉重写，编出不存在的逻辑——你拿到一份"看起来很合理但跟字节码没关系"的 Solidity 代码。

对的提法：

这段字节码从 PC=0 执行，请逐 PC 列出：opcode、执行后 stack([a,b,c] 栈顶在右)、改变的 memory(mem[off..len] = ...)、改变的 storage(slot k = ...)。JUMP/JUMPI 列出所有可能分支。完成后再给 Solidity 译文。

差别在哪？前一句让 AI "自由发挥"，后一句强制它老实模拟 EVM——它必须先把每一步算出来才能写答案，幻觉率显著下降。这是和 LLM 共事的通用技巧：把它从"想象家"按回"模拟器"。

CHAPTER 22

20 习题(含完整解答)

读完正文不一定真懂，自己算一遍才懂。下面六道题覆盖了 selector 反查、1559 费用、SSTORE 大表、CREATE2、字节码逆向、Yul vs Solidity——基本是 EVM 工程师面试常见题面，答案附在每题下面，自己先做再看答案。

20.1 题 1: 函数选择器逆向

给定 4-byte selector `0x70a08231`，请反查它对应的 Solidity 函数签名。

解答

`0x70a08231 = bytes4(keccak256("balanceOf(address)"))`。

```
cast keccak "balanceOf(address)" # → 0x70a08231...
```

不用 cast: 查 4byte.directory 或暴力字典枚举。

20.2 题 2: EIP-1559 与 legacy 费用差

设某区块 `baseFee = 50 gwei`，某用户发 legacy 交易愿付 `gasPrice = 70 gwei`，`gas_used = 50000`；同样的事换成 EIP-1559 (`maxFeePerGas = 70 gwei`, `maxPriorityFeePerGas = 5 gwei`)，用户实际付多少？被烧多少？

解答

Legacy: `70 gwei × 50000 = 0.0035 ETH`，全给矿工，无烧毁。

EIP-1559:

```
gasPrice = min(70, 50 + 5) = 55 gwei
fee = 55 × 50000 = 0.00275 ETH
burn = 50 × 50000 = 0.0025 ETH
tip = 5 × 50000 = 0.00025 ETH
```

用户付 0.00275 ETH(省 21.4%); 0.0025 烧毁, 0.00025 给提议者。

20.3 题 3: SSTORE gas

合约里有:

```
contract C {
    uint256 public x; // slot 0
    uint256 public y; // slot 1

    function f() external {
        x = 1;
        x = 2;
        y = x;
        y = 0;
    }
}
```

调用 `f()` 一共消耗多少 SSTORE 相关的 gas(不算其他 opcode、不算 intrinsic)? 给 refund。

假设 `x = 0`、`y = 0` 在调用前。

解答

1. `x = 1` : slot 0, `0→1`, cold 写: 22100, refund 0
2. `x = 2` : slot 0 已 warm, `1→2`, warm 非零→非零: 2900
3. `y = x` : SLOAD slot 0(warm 100); SSTORE slot 1, `0→2`, cold 写: 22100
4. `y = 0` : slot 1 已 warm, `2→0`, warm 清零: 2900, refund +4800

SSTORE 合计 = 22100 + 2900 + 22100 + 2900 = 50,000 gas; SLOAD = 100; refund = 4800(受 EIP-3529 截断: $\text{refund} \leq \text{gas_used} / 5$)。

20.4 题 4: CREATE2 地址预言

`deployer = 0x4e59b44847b379578588920ca78fbf26c0b4956c` (Foundry 默认 deterministic deployer);
`salt = 0x0000...0001`; `initCode = 0x6080604052348015..` (哈希为 `0x1234..` 共 32 字节, 假设你已知 `keccak256(initCode)`)。

请写出 CREATE2 地址的计算过程, 并给出步骤。

解答

```
addr = keccak256(0xff || deployer || salt || keccak256(initCode))[12:32]
```

`0xff` (1B) + `deployer` (20B) + `salt` (32B) + `initCodeHash` (32B) = 85 字节 → `keccak256` → 取低 20 字节。`0xff` 把 CREATE2 输入与 CREATE 的 RLP 格式区分开。

```
cast create2 \
  --deployer 0x4e59b44847b379578588920ca78fbf26c0b4956c \
  --salt 0x0000000000000000000000000000000000000000000000000000000000000001 \
  --init-code-hash 0x1234...
```

20.5 题 5: 字节码逆向(中等难度)

给定下面这段 runtime bytecode:


```
function hash(bytes memory data) external pure returns (bytes32) {
    return keccak256(data);
}
```

Yul(汇编):

```
function hashYul(bytes memory data) external pure returns (bytes32 result) {
    assembly {
        result := keccak256(add(data, 0x20), mload(data))
    }
}
```

两者 gas 几乎相同(差几十)——Solidity 内部就是这样编译的。Yul 的真正价值: 手写 dispatcher(省 fmp 开销)、packed encoding(绕开 32 字节对齐)、已知安全算术跳 overflow check、使用 EXTCODEHASH 等 Solidity 不直接暴露的 opcode。

CHAPTER 23

22 EOF 深入：被剔除又复活的 EVM 重整方案

EOF 是 EVM 历史最有戏剧性的提案。三年准备、九个 EIP、撑到了 Fusaka 纳入名单的最后一刻——然后 2025 年 9 月被整组剔除。

22.1 EOF 想解决什么

EOF 想干的事，听起来有点像 ELF 给 Linux 做的事：把"扁平的字节码"变成"带 header + section 的可执行对象文件"。

为什么需要？

- 现状：字节码扁平一锅粥，JUMPDEST 必须运行时检查，栈深也得每条指令复查——这些是 ZK 证明 EVM 时最贵的几项。
- EOF 之后：部署时一次校验，运行时不再检查。

听起来都是好事。但它撞上了 EIP-7702——后者抢用了 `0xEF` 这个本来给 EOF 预留的非法 opcode 前缀。Pectra 上线 EIP-7702 时为了赶进度、为了让 EOA 立刻能挂 designator，硬把 `0xef0100` 占用了。

Fusaka 拍板时 EOF 工程复杂度过高、devnet 多次同步失败、与 7702 的交互没充分论证——整组九个 EIP 一起被剔除。目前规划 Glamsterdam 之后再议，但有人私下说"再议"基本等于"重做"。

来源：[Tenderly: EVM Object Format \(EIP-7692\) 完整指南](#)、ethereum.org/roadmap/fusaka。

22.2 完整 EIP 清单("EOF Mega EIP" = EIP-7692)

EIP	名字	作用
EIP-3540	EOF v1	顶层格式：magic <code>0xEF00</code> + version + section header
EIP-3670	Code Validation	部署时静态校验 opcode 合法性、JUMPDEST 可达性
EIP-4200	Static relative jumps	RJUMP/RJUMPI/RJUMPV 替代动态 JUMP，去掉运行时 JUMPDEST 检查
EIP-4750	Functions	CALLF/RETF：真正的函数 ABI，分离调用栈与数据栈
EIP-5450	Stack Validation	部署时静态算每个 PC 的栈高度，运行时不再检查栈深
EIP-6206	JUMPF	函数尾调用，对应栈帧复用
EIP-7480	Data Section Access	DATACOPY/DATALOAD/DATALOADN/DATASIZE — 字节码内独立 data 段
EIP-663	DUPN/SWAPN	1-256 范围的 DUPn/SWAPn 替代固定 1-16
EIP-7620	EOFCREATE	替代 CREATE/CREATE2，强制部署 EOF 容器

22.3 EOF 的好处与代价

好处: 部署时一次校验, 运行时去掉 JUMPDEST 与栈深检查(ZK 友好); 函数 ABI 化省 ~200 gas dispatcher; code/data 分段携带常量; 静态 jump 提升反汇编准确率。

代价: 与 EIP-7702 的 `0xef0100` designator 前缀冲突(均源自 `0xEF` 协议预留); 工具链全面迁移(Solidity、Yul、Foundry、reth/geth); 与 stateless 的交互需重新论证。

22.4 与 EIP-7702 的尴尬关系

EIP-7702 designator = `0xef0100` || address (23 字节), `0xEF` 本是为 EOF 预留的非法 opcode 前缀。Pectra 抢先用了它, 使 EOF 必须重选 magic 或显式区分两种语义——是 EOF 被剔除的次要原因之一。

CHAPTER 24

23 Verkle Trees 深入：原理、过渡与 EIP-6800

§8 提了一句"Verkle 是给 MPT 做心脏外科手术"，这一节把刀拿出来真的开。

要点：MPT 的 Merkle 证明 1 KB / 槽，1000 个槽要 1 MB；Verkle 用椭圆曲线上的多项式承诺把同样的事压到 ~10 KB。这不是常数因子优化，是数量级——直接让"无状态客户端"从理论变成可能。

23.1 数学根基：vector commitment

核心：多项式承诺(Pedersen + IPA 在 Banderwagon 曲线上)。一个节点承诺 256 元素向量到一个 32 字节节点：

$$C = \sum (v_i \times G_i)$$

G_i 为 256 个独立曲线点。证"第 k 元素是 v_k "只需对数大小的 IPA 证明(~1 KB)，且可多点聚合——1000 个 (key, value) 对的总证明 ~10 KB，而 Merkle 需 1000 KB(不可聚合)。

23.2 EIP-6800：统一 Verkle 树结构

单一 Verkle 树取代 MPT 的多 storage trie。键空间：

```
key = pedersen_hash(address || treeIndex || subIndex)[31] || subIndex
```

所有字段(balance、nonce、codeHash、storage slot)映射到同一棵树的不同 (treeIndex, subIndex)。好处：单证明跨账户聚合；256 扇出，深 3-4；单证明 ~150 字节，全块 witness ~200 KB。

来源：[EIP-6800](#)、ethereum.org/roadmap/verkle-trees。

23.3 过渡机制：dual-tree

Hegota 升级时同时维护 MPT + Verkle：

1. To: 从 MPT 状态生成初始 Verkle root
2. To 后每块状态变更两边都写
3. 旧账户访问从 MPT 拉后写入 Verkle；新账户只在 Verkle
4. 约 6 个月后全量迁移，关闭 MPT

过渡开销：+25% 写入成本，+15% 磁盘。

23.4 Pectra 已做的 Verkle 准备工作

- EIP-2935: 8192 个区块哈希存进系统合约(Verkle 中取历史哈希用普通 SLOAD，非 BLOCKHASH opcode)
- EIP-7702 designator 用 `0xef01` 前缀(Verkle / EOF 预留前缀空间)

23.5 EIP-7736: leaf-level state expiry (依赖 Verkle)

Verkle extension-and-suffix 子树加访问计数器，过期 = 删除该子树。只有 Verkle 上线后方可实施，详见 §24。

CHAPTER 25

24 State expiry / History expiry / 弱无状态

以太坊状态 350 GB，每年还在涨。十年后呢？百年后呢？没人想跑 5 TB 的全节点。

但你不能简单“删掉旧数据”——总有人会回来问“2017 年那笔交易里我账户余额是多少”，总有合约睡了三年突然被人调用。三个相关方案分别在不同维度下手：history expiry 删历史块、state expiry 删冷账户、weak statelessness 干脆让普通节点根本不持有 state。它们不互斥，是叠在一起的瘦身组合拳。

24.1 三个概念

概念	它要解决的问题	实现复杂度
History expiry (EIP-4444)	节点不必永久保留过去所有块的 body	低(已部分实施)
State expiry	节点不必保留长期没人访问的账户/槽	高(需要 Verkle)
Weak statelessness	普通节点不必持有完整 state，只 builder 持有	中(依赖 Verkle + ePBS)

24.2 History expiry: EIP-4444

节点只保留最近一年的块 body 与 receipts；早期数据靠 portal network / IPFS / Arweave。已部分上线：geth/reth 默认 snap sync 不下载远古状态；portal network([Trin](#)、[Fluffy](#)、[Ultralight](#)) 2025 起提供部分轻验证。

24.3 State expiry

每个 storage slot 有上次访问时间戳，超过一年未访问则“过期”，节点可删除；复活时需附 Verkle 证明 witness。EIP-7736(leaf-level)最简方案：只删 Verkle 树叶节点(extension-and-suffix)，保留中间节点，粗估节省磁盘 50%+，不影响活跃合约。长期僵尸合约可由任何人附 witness 复活。

24.4 Weak statelessness

弱无状态：只 block builder 持完整 state，其他节点(attester / proposer / 用户全节点)拿 builder 提供的 witness 验证。依赖：Verkle Tree(witness $\leq$ ~200 KB) + ePBS(builder 角色协议化)。两者分别在 Hegota 和 Glamsterdam 落地，2027-2028 期间可期。

来源：ethereum.org/roadmap/statelessness、[Vitalik: state expiry roadmap](#)、[EIP-7736](#)。

CHAPTER 26

25 Lean Ethereum: beam chain 与后量子安全

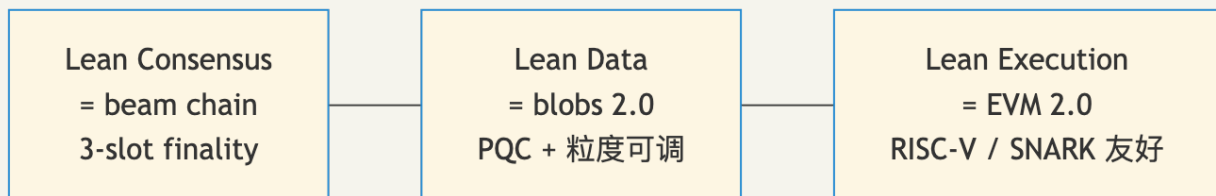
如果说 Pectra / Fusaka / Glamsterdam 是给以太坊"装新功能", 那 Lean Ethereum 是 Vitalik 在认真考虑"我们要不要把它整个重写一遍"。

动机有两条: 第一, 量子计算机如果哪天能破 secp256k1 和 BLS12-381, 今天链上所有签名都失效——必须提前换成 hash-based 后量子签名; 第二, 整个 L1 现在比 Bitcoin 复杂太多, 简化到能让一个本科生在一周内读完核心代码, 对长期安全性是必须的。

时间表非常远——最早 2028-2030——但方向已经定了。

25.1 路线图位置

2025-05 Vitalik "[Simplifying the L1](#)" 提出 Lean Ethereum——将 L1 简化至接近 Bitcoin 复杂度。三大组件:



来源: [Ethereum Foundation: lean Ethereum](#)、[leanroadmap.org](#)。

25.2 beam chain = Lean Consensus

替代当前 beacon chain: 3-slot finality (~36s, vs 当前 ~12.8min)、STARK 聚合签名替代 BLS attestation、Winternitz/XMSS hash-based 签名替代 BLS12-381(后量子安全), 核心逻辑约 200 行 Rust。

25.3 Lean Data = Blobs 2.0

- 粒度可调 blob: 固定 128 KB → 允许 32/64 KB 小 blob
- 后量子 KZG 替代: KZG 依赖椭圆曲线 trusted setup(量子可破), 将来用 FRI/STARK 风格 hash-based 承诺
- PeerDAS 持续演进降低带宽

25.4 Lean Execution = EVM 2.0

最激进: 把 EVM 整体替换成 RISC-V 或 ZK-prover 友好的 VM。理由: ZK rollup 内部已在把 EVM 翻译成 RISC-V 再证明, 为什么主网不直接跑 ZK 友好 VM? 代价: Solidity / Yul / Foundry 生态全面重对齐; 现有合约需 EVM-on-RISC-V 解释器(性能损失 1.5-3×)。时间表: 最早 2028-2030。

25.5 PQC(后量子)核心

当前: secp256k1 (EOA 签名) + BLS12-381 (验证者签名) + KZG (blob 承诺)
后量子: Winternitz/XMSS (hash-based) + STARK 聚合 + FRI 多项式承诺

hash-based 签名代价: 单签名 ~2-8 KB(vs ECDSA 64 字节), 但可深度聚合——Lean Consensus 目标把 1000 个 attestation 聚合到 ~50 KB STARK proof。

来源: [CryptoSlate: Vitalik's Lean Ethereum](#)。

CHAPTER 27

26 Reth 内部架构与 Execution Extensions (ExEx)

geth 是以太坊的"参考实现"——它定义对错。reth 是 Paradigm 写的"如果我今天从零写一个, 会怎么写"——结果是 1.7 Gigagas/s 的执行速度、240 GB 的磁盘占用、可以拆成 crate 当库用的代码结构。两年时间从 0 到 25% 主网份额。

这一节带你看它内部是怎么搭的。最有意思的是 ExEx——一种"插件机制", 让你不用 RPC poll 就能在节点里直接处理新块, 被 L2 sequencer、indexer、MEV 监控大量采用。

26.1 Reth 的设计哲学

"模块化、生产级、库优先"——每个组件可作独立 Rust crate。典型场景: L2 sequencer(import reth EVM + 状态库做自定义 rollup)、高吞吐索引器、ZK prover(reth trace 直接喂 ZK 电路)。

26.2 Reth 主要 crate

crate	作用
reth-evm	EVM 解释器, wrapper over revm
reth-db	MDBX 封装, 分 table 存账户/storage/header/body
reth-trie	MPT 实现(Verkle 在 reth-trie-verkle 里)
reth-network	devp2p / discv5 实现
reth-rpc	JSON-RPC + Engine API
reth-stages	staged sync 的每个阶段(headers / bodies / state / hash)
reth-exex	Execution Extensions 框架

26.3 Storage V2(Reth 2.0)

MDBX 完全重写: 1.7 Gigagas/s, 磁盘 ~240 GB(vs V1 ~340 GB), reth db migrate-v2 无须 resync。

26.4 Staged Sync(继承自 Erigon)

1. Headers — 下载所有块 header
2. Bodies — 下载所有块 body
3. SenderRecovery — ECDSA 恢复每笔 tx sender
4. Execution — 顺序执行更新状态
5. HashState — 扁平状态哈希成 trie
6. IntermediateHashes — 计算 stateRoot
7. Index — 构建查找索引

磁盘 IO 可预测, 支持 batch, 优雅断点续传。

26.5 Execution Extensions (ExEx)

编译进 reth 二进制的 plugin，每次状态提交时收到通知，无需 RPC poll。

```

async fn my_exex(mut ctx: ExExContext<Node>) → Result<(), ()> {
    while let Some(notification) = ctx.notifications.recv().await {
        match notification {
            ExExNotification::ChainCommitted { new } ⇒ {
                // 处理新块: indexer 写库 / rollup builder 派发 / MEV 监控
            }
            ExExNotification::ChainReorged { old, new } ⇒ { /* undo */ }
            ExExNotification::ChainReverted { old } ⇒ { /* undo */ }
        }
    }
    Ok(())
}

```

优势：共享 reth 状态内存（零延迟、零拷贝）；三种通知（ChainCommitted / ChainReorged / ChainReverted）保证 reorg 安全；编译时 type-checked。

实战：[reth-exex-rollup](#)（<1000 行实现 L2）、WeaveVM DA ExEx（Arweave 作 DA）、Subsquid/Goldskey 实时 indexer、Flashbots builder 模拟。

来源：reth.rs/exex/overview、[Quicknode: How to Build Reth ExEx](#)。

CHAPTER 28

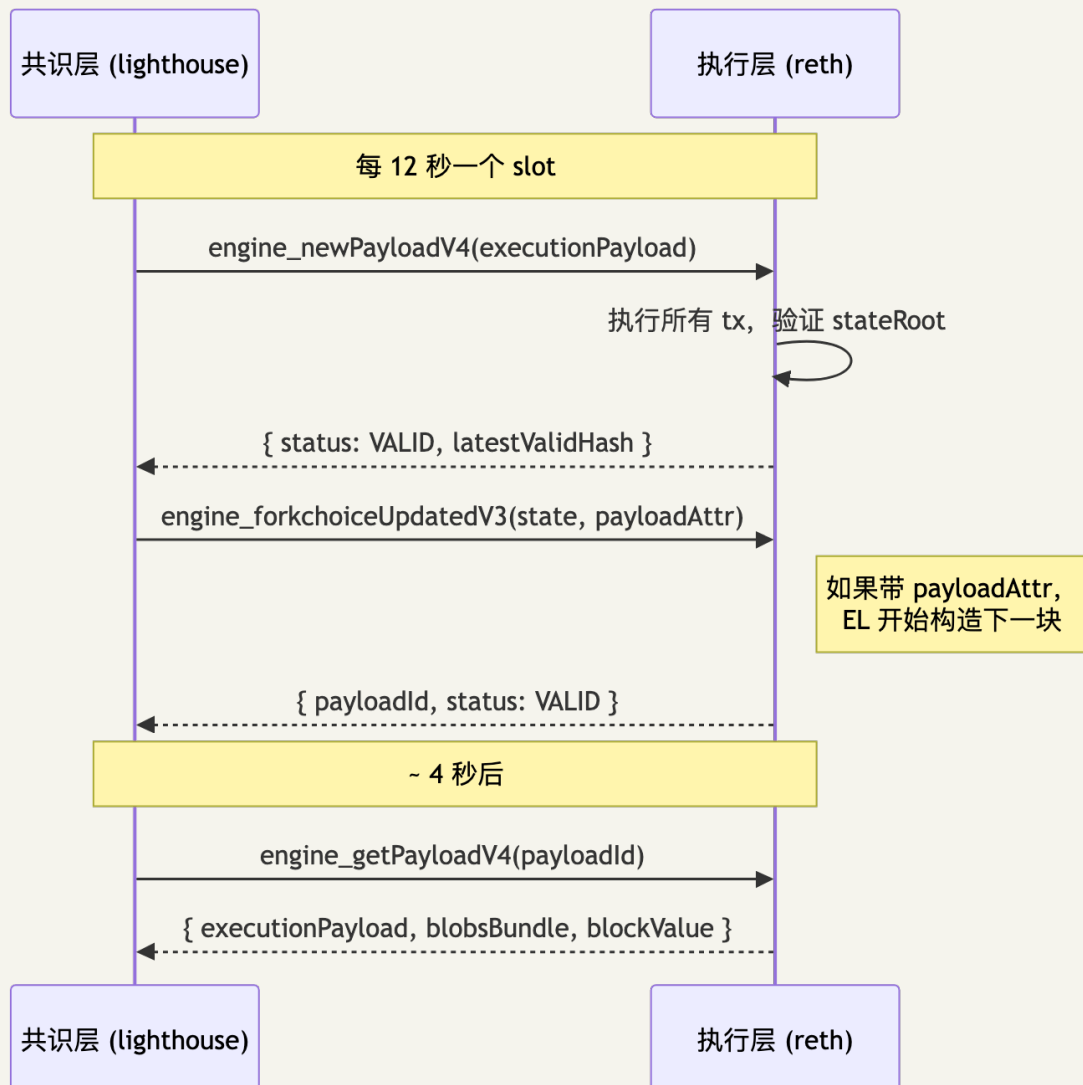
27 Engine API: 执行层与共识层之间的对话

§9 提到每台节点其实是两个进程——一个 EL、一个 CL，它们之间靠 Engine API 打电话。这一节是这通电话的逐字稿。

每 12 秒 CL 推一个新块给 EL 验证，EL 答 VALID / INVALID。每次 fork choice 改变 CL 通知 EL 调头部。EL 每 4 秒构造一次候选块给 CL 取走。理解这三个 RPC (newPayload / forkchoiceUpdated / getPayload) 就理解了节点上下层之间发生的几乎所有事情。

27.1 角色分工

CL 是"老板"(决定 head)，EL 是"工人"(执行块、维护状态)，通过 Engine API(HTTPS + JWT) 通信。



27.2 核心三方法

- `engine_newPayloadV4` (Pectra 后): CL 推 block 给 EL 验证, 返回 VALID / INVALID / SYNCING / ACCEPTED
- `engine_forkchoiceUpdatedV3`: CL 告知 head/safe/finalized 三块; 可选附 `payloadAttributes` 让 EL 开始构造下一块
- `engine_getPayloadV4`: CL 取回 EL 构造的 payload + blobs

27.3 JWT 认证

EL 与 CL 共享 32 字节 JWT secret, 防止外部进程冒充 CL。泄露 = 完全失控执行层。

```
openssl rand -hex 32 > /etc/ethereum/jwt.hex
chmod 600 /etc/ethereum/jwt.hex
# reth 与 lighthouse 都指向这同一个文件
```

27.4 不同 fork 对应的 V 版本

Fork	newPayload	forkchoiceUpdated	getPayload
Paris (Merge)	V1	V1	V1
Shanghai	V2 (+withdrawals)	V2	V2
Cancun	V3 (+blobVersionedHashes, parentBeaconBlockRoot)	V3	V3
Pectra	V4 (+executionRequests for deposit/ withdrawal/consolidation)	V3	V4
Fusaka	V4 + PeerDAS 字段	V3	V4 + data column sidecars

来源: [ethereum/execution-apis](https://ethereum.org/en/execution-apis/)、[Engine API: A Visual Guide](https://ethereum.org/en/execution-apis/visual-guide/)。

27.5 调试

```
# reth 启动加 --debug.engine-api, lighthouse 加 --debug-level info

curl -X POST http://localhost:8551 \
  -H "Authorization: Bearer $(jwt encode --secret @/etc/ethereum/jwt.hex)" \
  -H "Content-Type: application/json" \
  -d '{"jsonrpc":"2.0","method":"engine_getClientVersionV1","params":[{}],"id":1}'
```

CHAPTER 29

28 USDC 字节码 walkthrough (FiatTokenV2_2)

USDC 是主网最有钱的 ERC-20，市值常年 600 亿美元以上。它每天处理上百万笔 transfer——一个简单的 transfer 调用，背后是一段 2018 年的字节码、一个早于 EIP-1967 的 proxy 设计、一堆"先 SLOAD 黑名单再 SSTORE 余额"的合规检查。

第 12 节我们走完了它的 dispatcher。这一节走完它从 proxy 到 implementation 再回来的完整字节码路径，看看一笔"简单"的 USDC transfer 为什么要 60000 gas (普通 OZ ERC-20 只要 46000)。

28.1 链上结构

USDC 主网：0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606eB48 (代理)，当前 implementation：0x43506849D7C04F9138D1A2050bbF3A0c054402dd (FiatTokenV2_2, 2026-04-27 实测)。

使用 OZ Unstructured Storage (早于 EIP-1967 标准化)，slot：

```
ADMIN_SLOT      = keccak256("org.zeppelinos.proxy.admin")
                = 0x10d6a54a4754c8869d6886b5f5d7fbfa5b4522237ea5c60d11bc4e7a1ff9390b
IMPLEMENTATION_SLOT = keccak256("org.zeppelinos.proxy.implementation")
                = 0x7050c9e0f4ca769c69bd3a8ef740bc37934f8e2c036e5a723fd8ee048ed3f8c3
```

与 EIP-1967 (keccak256(...) - 1) 不同，USDC 部署在 EIP-1967 标准化之前。

来源：[Etherscan: USDC contract](#)、[Centre blog: Designing an upgradeable Ethereum contract](#)。

28.2 一笔 USDC transfer 的字节码路径

1. 进入 proxy fallback
2. proxy SLOAD ADMIN_SLOT → 检查 caller 是否是 admin
 - 是 admin: 走 admin_only 函数 (升级、转 admin)
 - 不是: 走 fallback → DELEGATECALL implementation
3. DELEGATECALL → FiatTokenV2_2.transfer()
 - 检查 paused (SLOAD slot 0x...)
 - 检查 from / to 不在 blacklist (mapping SLOAD)
 - SLOAD balances[msg.sender]
 - 余额检查
 - SSTORE balances[msg.sender] -= amount
 - SSTORE balances[to] += amount
 - LOG3 Transfer(from, to, amount)
4. RETURN 到 proxy
5. proxy RETURN 给用户

实测 USDC transfer ~63000 gas，比 OZ 简单 ERC-20 (~46000) 多 ~17000：2 个额外 SLOAD (paused + blacklist, 4200 gas) + proxy fallback/DELEGATECALL (~1500 gas) + SignatureChecker helper 加载。

28.3 USDC 的管制特性

OFAC 合规稳定币，链上管制接口(字节码可见)：

```
cast sig "pause()" # 0x8456cb59
cast sig "blacklist(address)" # 0xf9f92be4
cast sig "destroyBlackFundsAtAddress(address)" # USDC FiatTokenV2_2 销毁 blacklisted 余额
(注意: destroyBlackFunds 是 Tether USDT 的方法, USDC 用 destroyBlackFundsAtAddress)
```

28.4 为什么 USDC 不用 EIP-1967

USDC proxy 部署于 2018-09, 早于 EIP-1967(2019-04 标准化)。升级 implementation 无法改 proxy storage 习惯, 遂保留至今。新部署的 proxy 务必用 EIP-1967。读 USDC implementation 需用专属 slot:

```
cast storage 0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606eB48 \
0x7050c9e0f4ca769c69bd3a8ef740bc37934f8e2c036e5a723fd8ee048ed3f8c3
# 输出当前 implementation 地址
```

28.5 用 heimdall 反编译 FiatTokenV2_2

```
heimdallup
heimdall decompile 0x43506849D7C04F9138D1A2050bbF3A0c054402dd \
--rpc-url https://ethereum-rpc.publicnode.com \
--output ./usdc-decompiled
# 输出: decompiled.sol / abi.json / cfg.dot (graphviz 渲染)
```

--llm 选项(0.8+)做变量重命名与函数注释。对照 [circlefin/stablecoin-evm](https://github.com/circlefin/stablecoin-evm) 验证准确率。

CHAPTER 30

29 Pectra 一年后的现场观察

写在 2026-04, Pectra 上线 11 个月。这种"上线一年后回头看"的复盘往往比 EIP 提案本身更有信息量——提案画的饼是甜的, 现场吃下去是甜是苦得自己尝。

每个 EIP 都有它的现场剧本: EIP-7702 既造就了 MetaMask Smart Account, 也催生出第一周的钓鱼浪潮; EIP-7251 让大型 staking 把 1000 个 validator 合并成 30 个; EIP-7691 + Fusaka BPO 把 L2 swap 费从几美分压到零点几美分; EIP-7549 让 nimbus 重新能在树莓派上跑——还有那次 Prysm 事件, 给"客户端多样性"四个字提供了最贵的注脚。

29.1 EIP-7702 的现场情况

- 上线一周: 约 11,000 个 authorization, 主要来自 OKX、MetaMask、Ambire
- 上线一个月: MetaMask 默认引导用户升级至"Smart Account" (7702 + Safe 模板)
- 副作用: 上线两周内出现钓鱼合约, 诱导用户签向恶意 implementation; 报告称 97% authorization 关联钓鱼
- MetaMask 对策: UI 强制对照 implementation 代码哈希白名单 (OZ / Safe / Ambire 官方)

来源: [The Block: Smart wallet adoption surges](#)。

29.2 EIP-7251(MaxEB)的现场情况

- 上线时: ~2% staked ETH 来自 consolidated validator (>32 ETH Type-2)
- 6 个月后: consolidated 占比涨至 11%+, 约 7,000 个 Type-2 validator 持有 4M+ ETH
- 运营方: 管理数量从 1000+ validator 合并至 ~30 个, 成本降一个数量级
- slashing: 初始罚金从 1/32 降到 1/4096 (2048 ETH validator 被 slash 仅损 ~0.5 ETH)

来源: [Coinbase: Understanding MaxEB](#)。

29.3 EIP-7691(blob 上限)+ Fusaka BPO 的现场情况

时间	blob 目标	blob 上限	rollup 单 tx 成本(典型 swap)
Cancun (2024-03)	3	6	~\$0.05
Pectra (2025-05)	6	9	~\$0.025 (-50%)
Fusaka BPO1 (2025-12-09)	10	15	~\$0.012
Fusaka BPO2 (2026-01-07)	14	21	~\$0.005

L2 swap 费从几美分降至零点几美分; 每天 rollup blob 数从 ~21,300 (Cancun 末) 涨到 28,000+ (Pectra 后), 主网 calldata 让出 90%。

来源: [PANews: Blob Market After Pectra](#)。

29.4 EIP-7549(attestation 移 committee_index)

- BLS 验证次数减少 $\sim 64\times$ (每 epoch 从 ~ 200 万次降至 $\sim 30,000$ 次)
- finality 延迟降低 $\sim 30\%$, nimbus 重新可在树莓派运行

29.5 Prysm 事件复盘

2025-12-10(Fusaka 后 7 天), Prysm v6.0.4 在 PeerDAS 数据列处理路径有 race condition, $\sim 38\%$ validator attestation miss, finality 一度从 99.9% 降至 75%, 但没有 fork——其他四 CL 合计份额 $> 33\%$, 链没断。事件后 lighthouse / teku 份额各涨 $\sim 5\%$ 。客户端多样性是协议层防线而非营销话术, 正是这一节的实证。

CHAPTER 31

30 延伸阅读

- [Ethereum Yellow Paper](#) — 形式化定义的根源
- ethereum.org/developers/docs
- [evm.codes](#) opcode 与 precompile 速查
- [evm.codes/about](#) — 一篇极好的 EVM 入门长文
- [Geth Book](#)
- [Reth Book](#)
- [Pectra dev guide — Alchemy](#)
- [EIP-7702 站点](#)
- [EIP-2537 测试向量](#)
- [The State of Ethereum in 2026 — stake.fish](#)
- [Understanding the Yellow Paper's EVM Specifications](#)
- [Mastering Ethereum, Andreas M. Antonopoulos & Gavin Wood](#) Chapter 13: The Ethereum Virtual Machine
- [LearnEVM.com](#) — Solidity Developer 进阶 EVM 课程

下一步: [模块 04 — Solidity 开发](#)。